

函数卡 · 遇到再查，别背

建 2026.09.24 补 2026.09.28 补齐路线后半段 (LLM API / Agent / RAG / 数据 / FastAPI, +47 张)

卡数 126 张 (126 张已实测) 怎么用 搜名字 → 看场景 → 复制代码

这套卡**不是拿来背的** —— 是写代码卡住时用来查的。每张卡上的代码和输出，都由 `tools/build_fn_cards.py` 真的跑过一遍再写进来，所以卡里不会出现"看着合理、其实报错"的示例。

用法 这套卡怎么用（30 秒）

这套卡覆盖你路线的哪几段

路线段	卡数	为什么要有它
① 数据库（手上这件事） SQL / sqlite3	51	第 3 课 + D8 作业；所有后端岗的默认底子
② 语言与文件 标准库 / 内置	26	json / os / re / datetime —— 写任何脚本都绕不开
③ 项目 v1：调大模型 LLM API	15	2026.10 立项后的第一块砖；换一家模型只改两行
④ 项目 v1：Agent Agent	10	Agent = LLM + 上下文 + 工具（看板 2.1.1 已开线）
⑤ 项目 v1：检索增强 RAG	7	你已经跑到 Recall@1 87% 的那条线
⑥ 数据与算法调用 数据	8	pandas / numpy —— 物联网线每天要用
⑦ 对外提供接口 FastAPI	7	你的 api.py；服务器 8009 端口跑的就是它
⑧ 按需查 第三方	2	requests / pandas 等的签名速查

● 这张表是为了防止上一版的毛病：第一版 79 张里数据库占 65%，而 LLM API / Agent / RAG 一张都没有 —— 原因是我拿「这一周正在上的课」当了锚。现在改成按你的整条路线分段，哪一段空着，一眼就看得出来。

四个阶段标记怎么读

标记	意思
现在	手上这件事直接要用 —— 第 3 课 + D8 作业。其余的先不用看。
近期	这一两周会撞到（排序、聚合、文件、JSON、日期）。撞到再回来查。
阶段A	2026.10–2027.01 做项目 v1 要用的：大模型调用 / Agent / RAG / pandas / FastAPI。现在不用会，但可以翻着看，知道"原来就这几下"。
以后	2027 起才用得上的（正则进阶、第三方库签名）。看到名字能想起来就行。
实测输出（绿块）	脚本跑出来的真实结果，不是手写的。红块 = 故意让它报错的现场，因为报错信息本身就是要认的东西。

🖨️ 大模型那些卡（LLM API / Agent / RAG）是怎么"实测"的：全部跑在你本机的 Ollama 上（`qwen2.5:3b` + `bge-m3`，`127.0.0.1:11434`），不联网、不花钱。卡上的问答题是真调模型得到的那一次回答。⚠️ 但要说清楚：模型的回答每次会略有不同（`temperature=0` 也不保证逐字一样），所以你复跑时数字可能变 —— 这是模型的特性，不是卡写错了。

⚠️ 抄之前先改表名。SQL 那些卡跑在一份样板数据上（选修课三张表 `students / courses / enrollments` + 第 3 课的微信群三张表 `persons / groups / group_members`）—— 换成你自己的库，表名和列名都要跟着改。这正是第 3 课教的「换皮」。

内置函数不做成背诵卡（那条线已经关掉）—— 只留 8 张最常撞到的当查询用。要补哪些，说一声就加。

SQL 42 张

SELECT ... FROM ... 现在 SQL

所有查询的骨架：先说「要哪几列」，再说「从哪张表」

```
SELECT name, class FROM students;
```

实测输出

```
name | class
-----+-----
韩立 | 讯飞班
小明 | 讯飞班
小红 | 计科1班
小刚 | 计科1班
小美 | 计科1班
(5 行)
```

📌 第3课① · 任何一条 SQL 都长这样

⚠️ `SELECT *` 能跑但别常用 —— 列的顺序以后会变，代码会悄悄错位

WHERE 现在 SQL

先在脑子里选出全部，再一条条筛掉不要的

```
SELECT name, class FROM students WHERE class = '讯飞班';
```

实测输出

```
name | class
-----+-----
韩立 | 讯飞班
小明 | 讯飞班
(2 行)
```

📌 第3课 · 筛选；D8 作业至少要用一次

⚠️ 筛完只剩 2 行不是因为「表里只有 2 行」，是 WHERE 把它们筛掉了

ON / WHERE 分工 现在 SQL

ON = 怎么配对（横着拼），WHERE = 要哪些（竖着筛）

```
SELECT s.name, e.cid
FROM students s
JOIN enrollments e ON s.id = e.sid
WHERE e.cid = 1;
```

实测输出

name	cid
韩立	1
小明	1

(2 行)

第3课④ · 全课最该分清的一对

⚠ 把 `e.cid = 1` 挪到 ON 里，INNER JOIN 结果一样——但换成 LEFT JOIN 就天差地别（留在 ON 里 = 右表筛完再拼，放 WHERE 里 = 拼完再筛，会把 NULL 行筛掉）

COUNT(*) / COUNT(列) 现在 SQL

COUNT(*) 数行；COUNT(列) 只数**这一列不是 NULL**的行

```
SELECT COUNT(*) AS rows_all, COUNT(grade) AS
rows_graded FROM enrollments;
```

实测输出

rows_all	rows_graded
6	5

(1 行)

第3课⑥ 诊断第一步：单独数绳子表

⚠ 两者不一样就说明这列有 NULL——这个差值本身就是线索，不是 bug

JOIN ... ON（两表） 现在 SQL

拿一边的 id 去另一边配对，配对成功才出一行

```
SELECT s.name, e.cid
FROM students s
JOIN enrollments e ON s.id = e.sid;
```

实测输出

name	cid
韩立	1
韩立	2
小明	1
小明	3
小红	5
小刚	4

(6 行)

第3课④ · 本课核心

⚠ 结果行数 = 配对成功的次数，不是任何一张表的行数

三表 JOIN（中间那张是「绳子」） 现在 SQL

行数由中间那张绳子表说了算；两头都只是「换名字」

```
SELECT s.name AS student, c.name AS course,
e.grade
FROM students s
JOIN enrollments e ON s.id = e.sid
JOIN courses      c ON e.cid = c.id;
```

实测输出

student	course	grade
韩立	数据库	88.5
韩立	高数	76.0
小明	数据库	92.0
小明	机器学习	81.0
小红	英语	59.0
小刚	体育	NULL

(6 行)

第3课④ · 第6章换皮就是把它改名

⚠ 三张表的顺序可以换，但**每一句 ON 必须紧跟着它要拼的那张表**

坑：两边都有 id，不给表起别名 现在 SQL

两张表都有 `id` 列时，`ON students.id = courses.id` 里的 id 到底是谁的？

```
SELECT id, name FROM students JOIN courses ON
students.id = courses.id;
```

实测：真的会这样报错

OperationalError: ambiguous column name: id

报错现场——别名就是为了解决它

⚠ `ambiguous column name` 的正确解法是给表起别名然后写全：`s.id`、`c.id`

坑②：ON 接反 → 0 行（不报错） 现在 SQL

`groups.id` 是 1/2/3，`group_members.uid` 是 7/9/7——一个都对不上

```
SELECT persons.name AS person, groups.name AS
grp
FROM groups
JOIN group_members ON groups.id =
group_members.uid
JOIN persons      ON group_members.gid =
persons.id;
```

实测输出

person	grp
--------	-----

(0 行)

第3课④ 实测反例 1（低危：空的一眼看得出来）

⚠ JOIN 查出空表时的第一反应应该是「怀疑 ON」，而不是「数据没了」

坑②: **id 重叠** → 照样出数据 (最危险) 现在 SQL

students.id 和 courses.id 都从 1 开始, 接反了照样配得上——于是给你一张看着完全合理的假表

```
SELECT s.name AS student, c.name AS course
FROM students s
JOIN enrollments e ON s.id = e.cid
JOIN courses      c ON e.sid = c.id;
```

实测输出

student	course
韩立	数据库
小明	数据库
韩立	高数
小红	高数
小美	机器学习
小刚	体育

(6 行)

📌 第3课④ 实测反例 2 (🔴 高危: 静默错误)

⚠️ 「不为空」**绝不等于**「对」。唯一的防线是**挑一行人工核对内容**——这张表里有**根本没选课的小美**、还有对不上课的小红, 一眼就能看出不对

证明它到底错在哪: 行数一样, 内容不同 现在 SQL

同一批数据, 正确写法和接反写法**都出 6 行**; 真正差的是其中 2 行的内容

```
SELECT
  (SELECT COUNT(*) FROM (
    SELECT s.name, c.name FROM students s
    JOIN enrollments e ON s.id = e.sid JOIN
    courses c ON e.cid = c.id
    EXCEPT
    SELECT s.name, c.name FROM students s
    JOIN enrollments e ON s.id = e.cid JOIN
    courses c ON e.sid = c.id
  )) AS missing_in_wrong,
  (SELECT COUNT(*) FROM (
    SELECT s.name, c.name FROM students s
    JOIN enrollments e ON s.id = e.cid JOIN
    courses c ON e.sid = c.id
    EXCEPT
    SELECT s.name, c.name FROM students s
    JOIN enrollments e ON s.id = e.sid JOIN
    courses c ON e.cid = c.id
  )) AS rows_that_are_fake;
```

实测输出

missing_in_wrong	rows_that_are_fake
2	2

(1 行)

📌 第3课④ 那张假数据卡的下半句 —— 用 SQL 自己证明

⚠️ 左边 = 正确结果里有、接反结果里没有的 (漏了 2 行); 右边 = 接反结果凭空多出来的 (假了 2 行)。⇒ 只数行数永远发现不了, 必须看内容

诊断三步: 先数绳子, 再数结果 现在 SQL

① 单独数绳子表 ② 数 JOIN 结果 ③ 两数一致不算过关, 再挑一行核对内容

```
SELECT (SELECT COUNT(*) FROM enrollments) AS
rope_rows,
      (SELECT COUNT(*) FROM students s JOIN
enrollments e ON s.id = e.sid) AS join_rows;
```

实测输出

rope_rows	join_rows
6	6

(1 行)

📌 第3课⑥ · 本课最实用的迁移技能

⚠️ 数字对不上 (空 / 多很多) → 几乎一定是 ON; 数字**对得上**也可能内容错 (id 重叠时)

LEFT JOIN 现在 SQL

左表一行都不能丢; 右表配不上就整行填 NULL

```
SELECT s.name, e.cid, e.grade
FROM students s LEFT JOIN enrollments e ON
s.id = e.sid;
```

实测输出

name	cid	grade
韩立	1	88.5
韩立	2	76.0
小明	1	92.0
小明	3	81.0
小红	5	59.0
小刚	4	NULL
小美	NULL	NULL

(7 行)

📌 第3课⑤ 换皮3 · 统计时最常见

⚠️ 把 **LEFT** 去掉, **没选课的人会整个消失** —— 人数从 5 变 4, 而且不报错

INNER vs LEFT: 差在哪一行 现在 SQL

同一条查询, 只差 LEFT 四个字母, 结果少一行

```
SELECT s.name FROM students s JOIN enrollments
e ON s.id = e.sid;
```

实测输出

```
name
----
韩立
韩立
小明
小明
小红
小刚
(6 行)
```

📌 对照上一张卡看: 5 个人变成了 4 个人, 小美没了

⚠️ 要「一个都不漏」的清单 (点名、对账、覆盖率) 时必须 LEFT

看这个库有哪些表 现在 SQL

不用装数据库工具, 一条 SQL 就能看清家底

```
SELECT name FROM sqlite_master WHERE type =
'table' ORDER BY name;
```

实测输出

```
name
-----
courses
enrollments
group_members
groups
persons
students
(6 行)
```

📌 第3课①「这库里一共几张表」那道题的正确做法

⚠️ sqlite_master 是 SQLite 专有; MySQL 里叫 SHOW TABLES

看一张表的结构 现在 SQL

列名、类型、是否主键, 一条 PRAGMA 全给你

```
PRAGMA table_info(enrollments);
```

实测输出

```
cid | name | type | notnull | dflt_value |
pk
-----+-----+-----+-----+-----
+---
0 | sid | INTEGER | 0 | NULL |
0
1 | cid | INTEGER | 0 | NULL |
0
2 | grade | REAL | 0 | NULL |
0
(3 行)
```

📌 忘了列名时先看这个, 别猜

⚠️ PRAGMA 不是标准 SQL (且不吃 ? 占位符) —— 表名只能拼字符串, 只用于自己写死的表名

CREATE TABLE 现在 SQL

并列三件事: 列定义、主键、外键约束

```
CREATE TABLE enrollments_new (
    sid INTEGER NOT NULL REFERENCES
students(id),
    cid INTEGER NOT NULL REFERENCES
courses(id),
    grade REAL,
    PRIMARY KEY (sid, cid)
);
```

实测输出

```
cid | name | type | notnull | dflt_value |
pk
-----+-----+-----+-----+-----
+---
0 | sid | INTEGER | 1 | NULL |
1
1 | cid | INTEGER | 1 | NULL |
2
2 | grade | REAL | 0 | NULL |
0
(3 行)
```

📌 第3课③ · D8 作业的第一件事

⚠️ **复合主键** PRIMARY KEY (sid, cid) 直接防住「同一个人重复选同一门课」—— 比在 Python 里查一遍再插可靠

INSERT INTO 现在 SQL

列名写全，值用 ? 占位（Python 里）

```
INSERT INTO students (id, name, class) VALUES
(6, '小七', '计科1班');
```

实测输出

```
id | name | class
---+-----+-----
6  | 小七 | 计科1班
(1 行)
```

📌 D8 作业：插 3 行数据

⚠️ 列名和值数量必须一致，否则报 `table has N columns but M values were supplied`

ORDER BY 近期 SQL

排序；默认升序，DESC 反过来

```
SELECT s.name, e.grade
FROM enrollments e JOIN students s ON s.id =
e.sid
ORDER BY e.grade DESC;
```

实测输出

```
name | grade
-----+-----
小明 | 92.0
韩立 | 88.5
小明 | 81.0
韩立 | 76.0
小红 | 59.0
小刚 | NULL
(6 行)
```

📌 出榜单 / 看最大的那个

⚠️ NULL 在 SQLite 里排在最前（DESC 时排最后）—— 想让它垫底要写 `ORDER BY grade IS NULL, grade DESC`

LIMIT / OFFSET 近期 SQL

只要前 N 条 / 跳过前 N 条（分页就靠它）

```
SELECT name, credit FROM courses ORDER BY
credit DESC LIMIT 2;
```

实测输出

```
name | credit
-----+-----
高数 | 5
数据库 | 3
(2 行)
```

📌 Top-N 排行榜；翻页

⚠️ LIMIT 没有 ORDER BY 时顺序是未定义的 —— 每次跑可能不一样

DISTINCT 近期 SQL

一行里多个列都相同才算重复，去重后只剩一种

```
SELECT DISTINCT class FROM students;
```

实测输出

```
class
-----
讯飞班
计科1班
(2 行)
```

📌 「有几种」而不是「有几条」

⚠️ `COUNT(DISTINCT 列)` 才是「数几种」；`COUNT(*)` 数的是行数

AND / OR 的优先级 近期 SQL

AND 比 OR 先算 —— 所以 OR 两边一定要加括号

```
SELECT name, class FROM students
WHERE (class = '讯飞班' OR class = '计科1班')
AND name <> '小红';
```

实测输出

```
name | class
-----+-----
韩立 | 讯飞班
小明 | 讯飞班
小刚 | 计科1班
小美 | 计科1班
(4 行)
```

📌 多条件筛选

⚠️ 不加括号会变成「讯飞班的人，或者（计科1班且不叫小红的人）」—— 结果多出来的人你还不一定看得出来

IN 近期 SQL

一个字段对一串值，比写一堆 OR 干净

```
SELECT name FROM students WHERE class IN ('讯飞班', '计科1班');
```

实测输出

```
name
-----
韩立
小明
小红
小刚
小美
(5 行)
```

📌 按一批 id 批量查（爬虫/接口返回的 id 列表）

⚠️ NOT IN 遇到 NULL 会整体变空 —— 子查询里可能有 NULL 时别用

BETWEEN 近期 SQL

区间筛选，**两头都算在内**

```
SELECT name, credit FROM courses WHERE credit
BETWEEN 2 AND 5;
```

实测输出

name	credit
数据库	3
高数	5
机器学习	2
英语	2

(4 行)

按时间/分数区间查

⚠ BETWEEN 2 AND 5 = >= 2 AND <= 5 (含 5) —— 差一个边界是常见 off-by-one

LIKE 通配 近期 SQL

% = 任意多个字符，_ = 正好一个字符

```
SELECT id, name FROM courses WHERE name LIKE
'_数';
```

实测输出

id	name
2	高数

(1 行)

模糊搜索 / 清洗数据时找脏值

⚠ SQLite 的 LIKE 对 ASCII **大小写不敏感**，对中文当然无所谓 —— 但换成 PostgreSQL 就敏感了

COUNT(DISTINCT 列) 近期 SQL

数「有几种」，不是「有几条」

```
SELECT COUNT(*) AS rows_all, COUNT(DISTINCT
cid) AS kinds FROM enrollments;
```

实测输出

rows_all	kinds
6	5

(1 行)

统计去重后的数量（多少门课被选过）

⚠ COUNT(DISTINCT a, b) 在 SQLite 里是**语法错误**，多列去重得用子查询

SUM / AVG / MIN / MAX / ROUND 近期 SQL

五个聚合函数，NULL 一律不参与计算

```
SELECT ROUND(AVG(grade), 1) AS avg_g,
MAX(grade) AS top, MIN(grade) AS low
FROM enrollments WHERE grade IS NOT NULL;
```

实测输出

avg_g	top	low
79.3	92.0	59.0

(1 行)

算总分/均分；第7章预习要用

⚠ AVG 忽略 NULL，所以分母是「有分数的行数」而不是总行数 —— 拿不准就先 COUNT(*) 和 COUNT(列) 都打出来看

GROUP BY 近期 SQL

把行按某列分成堆，每堆出一行（聚合函数的分母就变成「这一堆」）

```
SELECT cid, COUNT(*) AS n FROM enrollments
GROUP BY cid ORDER BY n DESC;
```

实测输出

cid	n
1	2
5	1
4	1
3	1
2	1

(5 行)

第7章预习 · 每个群的成员数这种「分堆统计」

⚠ SELECT 里只能出现**分组列 + 聚合函数**，出现别的列时 SQLite 不报错但会随便挑一行给你（别的数据库直接报错）

HAVING 近期 SQL

筛「堆」。WHERE 筛行、发生在分组前，HAVING 筛堆、发生在分组后

```
SELECT cid, COUNT(*) AS n FROM enrollments
GROUP BY cid HAVING COUNT(*) > 1;
```

实测输出

cid	n
1	2

(1 行)

第7章预习 · 「选课人数 > 1 的课」

⚠ 写成 WHERE COUNT(*) > 1 会直接报错 —— 因为 WHERE 执行时还没分组，没有「每堆」这个概念

坑：WHERE 里用聚合函数 近期 SQL

WHERE 比 GROUP BY 先执行，所以它根本看不到 COUNT(*)

```
SELECT cid FROM enrollments WHERE COUNT(*) > 1
GROUP BY cid;
```

实测：真的会这样报错

OperationalError: misuse of aggregate: COUNT()

📌 报错现场

⚠️ 看到 `misuse of aggregate function` 就一个动作：把条件从 WHERE 挪到 HAVING

IFNULL / COALESCE 近期 SQL

NULL 换个默认值再显示（NULL 参与算术会让整行变 NULL）

```
SELECT sid, cid, IFNULL(grade, 0) AS grade
FROM enrollments;
```

实测输出

sid	cid	grade
1	1	88.5
1	2	76.0
2	1	92.0
2	3	81.0
3	5	59.0
4	4	0

(6 行)

📌 配 LEFT JOIN 用：没匹配上的那侧全是 NULL

⚠️ 别用 `IFNULL(grade,0)` 去「修」平均值——用 0 冒充「没考」会把均分拉低，该用 `WHERE grade IS NOT NULL`

坑：`= NULL` 永远查不出东西 近期 SQL

NULL 的意思是「不知道」，所以「不等于不知道」的答案是「我也不知道」

```
SELECT sid, cid FROM enrollments WHERE grade =
NULL;
```

实测输出

sid	cid
-----	-----

(0 行)

📌 查「没填的那几条」时最常踩

⚠️ 要判 NULL 只能用 `IS NULL` / `IS NOT NULL`——这个坑不报错、只是静默返回 0 行

IS NULL / IS NOT NULL 近期 SQL

判断「这格是不是空的」唯一正确写法

```
SELECT sid, cid FROM enrollments WHERE grade
IS NULL;
```

实测输出

sid	cid
4	4

(1 行)

📌 找脏数据 / 找漏填的记录

⚠️ 反过来记住：`WHERE grade <> 80` 也不会把 NULL 的行选出来（NULL 不满足任何比较）

UPDATE ... WHERE 近期 SQL

改数据。没有 WHERE 就是改全表（没有撤销）

```
UPDATE enrollments SET grade = 90 WHERE sid =
3 AND cid = 5;
```

实测输出

sid	cid	grade
3	5	90.0

(1 行)

📌 补分数、改状态

⚠️ 养成习惯：先把 `WHERE` 那段单拎出来 `SELECT` 一遍，确认命中行数，再改成 UPDATE

DELETE FROM ... WHERE 近期 SQL

删行。同样：没有 WHERE 就是清空整张表

```
DELETE FROM students WHERE id = 5;
```

实测输出

left_rows
4

(1 行)

📌 删脏数据

⚠️ 删之前先 `SELECT COUNT(*)`；真删表用 `DROP TABLE`（连表结构一起没）

CASE WHEN 以后 SQL

SQL 里的 if-else, 可以放在 SELECT 里造一个新列

```
SELECT name, CASE WHEN credit >= 3 THEN
'major' ELSE 'minor' END AS weight
FROM courses;
```

实测输出

name	weight
数据库	major
高数	major
机器学习	minor
体育	minor
英语	minor

(5 行)

📌 给数据打标签 (分数段/等级), 打标后直接喂给模型

⚠️ 不写 ELSE 时默认是 NULL, 不是原值

CAST 以后 SQL

改类型。SQLite 类型松, 但拼接/比较时该转还得转

```
SELECT CAST('42' AS INTEGER) + 1 AS n,
CAST(3.7 AS INTEGER) AS i;
```

实测输出

n	i
43	3

(1 行)

📌 从文本里提出来的数字要参与计算时

⚠️ CAST(3.7 AS INTEGER) 是截断不是四舍五入 (得 3); 要四舍五入用 ROUND

UPPER / LOWER / LENGTH 以后 SQL

大小写与长度。注意 LENGTH 数的是**字符**, 不是字节

```
SELECT UPPER('abc') AS u, LOWER('ABC') AS l,
LENGTH('数据库') AS n;
```

实测输出

u	l	n
ABC	abc	3

(1 行)

📌 数据清洗; 校验字段长度

⚠️ LENGTH('数据库') = 3 (SQLite 数字符), 但 LENGTH(CAST('数据库' AS BLOB)) = 9 —— 中文是 3 字节

SUBSTR / REPLACE / TRIM 以后 SQL

截取、替换、去首尾空格 —— 清洗三件套

```
SELECT SUBSTR('2026-09-24', 1, 4) AS y,
REPLACE('a-b', '-', '/') AS r, TRIM(' x ')
AS t;
```

实测输出

y	r	t
2026	a/b	x

(1 行)

📌 日志/网页文本清洗

⚠️ SQLite 的 SUBSTR 下标从 1 开始 (Python 从 0 开始), SUBSTR(s, 1, 4) 才是钱 4 个字符

date('now') / strftime 以后 SQL

SQLite 没有专门的日期类型, 日期就是文本, 靠这些函数算

```
SELECT date('now') AS today, strftime('%Y-%m',
'now') AS ym,
CAST(julianday('now') -
julianday('2026-09-21') AS INTEGER) AS days;
```

实测输出

today	ym	days
2026-09-28	2026-09	7

(1 行)

📌 按天统计; 算「距今几天」

⚠️ date('now') 取的是 UTC —— 想按北京时间算要写 date('now', '+8 hours')

GROUP_CONCAT 以后 SQL

把一堆行的某列拼成一个字符串 (默认用逗号)

```
SELECT cid, GROUP_CONCAT(sid) AS sids FROM
enrollments GROUP BY cid;
```

实测输出

cid	sids
1	1,2
2	1
3	2
4	4
5	3

(5 行)

📌 「这个群都有谁」 —— 一行拿到全部成员

⚠️ 拼接顺序不保证; 要固定顺序得先在外面排序 (SQLite 3.44+ 才支持 GROUP_CONCAT(x ORDER BY y))

子查询 以后 SQL

把一条 SELECT 的结果当另一条的条件用

```
SELECT name FROM courses
WHERE id IN (SELECT cid FROM enrollments WHERE
grade > 85);
```

实测输出

```
name
-----
数据库
(1 行)
```

🔴 两步查询懒得写两遍时

⚠️ `IN` (子查询) 里如果出现 `NULL`, `NOT IN` 会整体返回空
—— 拿不准就用 `EXISTS`

UNION 以后 SQL

上下摆两条结果集; UNION 去重, UNION ALL 不去重 (也更快)

```
SELECT name FROM students WHERE class = '讯飞班'
UNION
SELECT name FROM students WHERE id <= 3;
```

实测输出

```
name
----
小明
小红
韩立
(3 行)
```

🔴 合并两张结构相同的表 (比如两个月的日志)

⚠️ 两边**列数必须一样**, 列名以第一条为准 —— 列对不齐时不报错, 只会悄悄错位

sqlite3 9 张

六行骨架: 连接 → 游标 → 执行 → 取 → 关 现在

sqlite3

连 Python 和数据库的全部动作就这六行

```
import sqlite3
conn = sqlite3.connect(":memory:") #
# 文件路径: :memory: = 内存库
cur = conn.cursor()
cur.execute("CREATE TABLE t (x)")
cur.execute("INSERT INTO t VALUES (1), (2)")
conn.commit() #
# 写操作必须 commit
for row in cur.execute("SELECT x FROM t"):
    print(row)
conn.close()
```

实测输出

```
(1,)
(2,)
```

🔴 第3课 · 所有 SQL 作业的外壳

⚠️ `commit()` 忘写的后果见下一张卡 —— 它和 `close()` 是两件事

`commit()` vs `close()`: 谁在丢数据 现在 sqlite3

P-014 的实测: 换一个连接去读, 看数据还在不在

```
import sqlite3, tempfile, os, shutil
d = tempfile.mkdtemp(); p = os.path.join(d, "k.db")
```

```
c = sqlite3.connect(p); c.execute("CREATE TABLE t(x)")
c.execute("INSERT INTO t VALUES (1)")
c.close() #
# 只 close, 不 commit
c2 = sqlite3.connect(p)
print("commit X / close ✓ → 另一个连接读到",
c2.execute("SELECT COUNT(*) FROM t").fetchone()[0], "行")
c2.close()
```

```
c = sqlite3.connect(p); c.execute("INSERT INTO t VALUES (2)")
c.commit(); c.close() #
# 先 commit, 再 close
c2 = sqlite3.connect(p)
print("commit ✓ / close ✓ → 另一个连接读到",
c2.execute("SELECT COUNT(*) FROM t").fetchone()[0], "行")
c2.close(); shutil.rmtree(d, ignore_errors=True)
```

实测输出

```
commit X / close ✓ → 另一个连接读到 0 行
commit ✓ / close ✓ → 另一个连接读到 1 行
```

🔴 第3课① 检索题 (当场答错的那道) · P-014

⚠️ 丢数据的凶手是 `commit`, 不是 `close`。 `close()` 忘写的代价是占资源 (见下张卡), 不是丢数据

close() 忘了写会怎样 现在 sqlite3

连接不关 → 文件被锁住 → 连删都删不掉

```
import sqlite3, tempfile, os, shutil
d = tempfile.mkdtemp(); p = os.path.join(d, "k.db")
c = sqlite3.connect(p); c.execute("CREATE TABLE t(x)"); c.commit()
try:
    os.remove(p) #
    连接还开着, Windows 不让删
    print("删掉了 (没锁)")
except PermissionError as e:
    print("PermissionError:", e)
c.close()
os.remove(p) #
    关掉之后再删就成功了
print("close 之后再删: 成功")
shutil.rmtree(d, ignore_errors=True)
```

实测输出

```
PermissionError: [WinError 32] 另一个程序正在使用
此文件, 进程无法访问。:
'E:\\Temp\\tmptwk_r4_e\\k.db'
close 之后再删: 成功
```

📌 P-014 的另一半: close 的真实代价

⚠️ Linux/Mac 上删得掉, 只有 Windows 会当场报 WinError 32 —— 所以这个坑在 Windows 上反而更容易被发现

fetchone / fetchall / 直接遍历 现在 sqlite3

三种取结果的方式, 选中一条就不可能再回头

```
import sqlite3
conn = sqlite3.connect(":memory:")
conn.execute("CREATE TABLE t(x)")
conn.executemany("INSERT INTO t VALUES (?)",
[[1,], (2,), (3,)])
cur = conn.cursor()
cur.execute("SELECT x FROM t")
print("fetchone →", cur.fetchone()) #
    拿走第 1 行, 游标前移
print("fetchall →", cur.fetchall()) #
    剩下的全拿走
cur.execute("SELECT x FROM t")
print("遍历 →", [r[0] for r in cur])
conn.close()
```

实测输出

```
fetchone → (1,)
fetchall → [(2,), (3,)]
遍历 → [1, 2, 3]
```

📌 取查询结果

⚠️ 游标是**一次性的**: fetchone() 之后再 fetchall() 拿不到已经取走的那行。要重复用就把结果存进 list

executemany() 批量插入 现在 sqlite3

一次插 20 行, 别写 20 条 execute

```
import sqlite3
conn = sqlite3.connect(":memory:")
conn.execute("CREATE TABLE s(name TEXT, score REAL)")
rows = [("小明", 88.5), ("小红", 92.0), ("小刚", 76.5)]
conn.executemany("INSERT INTO s VALUES (?, ?)", rows)
conn.commit()
print(conn.execute("SELECT COUNT(*) FROM s").fetchone()[0], "行")
for r in conn.execute("SELECT * FROM s ORDER BY score DESC"):
    print(r)
conn.close()
```

实测输出

```
3 行
('小红', 92.0)
('小明', 88.5)
('小刚', 76.5)
```

📌 灌测试数据 / 批量写爬下来的数据

⚠️ 参数是一个可迭代对象, 每项对应一条语句; 写成 executemany(sql, (1,2)) 会把它当成两行单列

`?` 占位符 (防注入) 现在 sqlite3

值永远走参数, 不要用 f-string 拼进 SQL

```
import sqlite3
conn = sqlite3.connect(":memory:")
conn.execute("CREATE TABLE t(name TEXT)");
conn.execute("INSERT INTO t VALUES ('小明')")
who = "小明" OR '1'='1' #
    想注入的人会这么输
n = conn.execute("SELECT COUNT(*) FROM t WHERE name = ?", (who,)).fetchone()[0]
print("参数化 (对) →", n, "行")
bad = "SELECT COUNT(*) FROM t WHERE name = '%s'" % who
n2 = conn.execute(bad).fetchone()[0]
print("拼字符串 (错) →", n2, "行")
conn.close()
```

实测输出

```
参数化 (对) → 0 行
拼字符串 (错) → 1 行
```

📌 所有带用户输入的查询

⚠️ **单元素也必须写成** (who,) —— 少了逗号 (who) 只是个字符串, 会被当成一个可迭代对象逐字符拆开, 报 Incorrect number of bindings

row_factory: 按列名取值 **现在** `sqlite3`

设一次 `row_factory`，之后 `row['name']` 就能用

```
import sqlite3
conn = sqlite3.connect(":memory:")
conn.row_factory = sqlite3.Row          # ← 就这一行
conn.execute("CREATE TABLE s(id INTEGER, name TEXT)")
conn.execute("INSERT INTO s VALUES (1, '小明')")
row = conn.execute("SELECT id, name FROM s").fetchone()
print("按下标 →", row[0], row[1])
print("按列名 →", row['id'], row['name'])
print("变字典 →", dict(row))
conn.close()
```

实测输出

按下标 → 1 小明
按列名 → 1 小明
变字典 → {'id': 1, 'name': '小明'}

📌 结果列一多，下标就记不住了

⚠️ `row_factory` 设在 **connection** 上，不是 cursor 上（设 cursor 上是另一套写法）

with: 自动 **commit / rollback** **以后** `sqlite3`

`with conn:` 出块时自动提交，出异常自动回滚 —— **但不会关连接**

```
import sqlite3
conn = sqlite3.connect(":memory:")
with conn:                                #
    conn.execute("CREATE TABLE t(x)")
    conn.execute("INSERT INTO t VALUES (1)")
try:
    with conn:                            #
        conn.execute("INSERT INTO t VALUES (2)")
        raise ValueError("假装中途出错")
except ValueError as e:
    print("捕获到: ", e)
print("表里剩", conn.execute("SELECT COUNT(*) FROM t").fetchone()[0], "行")
conn.close()
```

实测输出

捕获到: 假装中途出错
表里剩 1 行

📌 写多步操作时不想每步都 commit

⚠️ `with conn:` ≠ `with sqlite3.connect() as conn:`。后者出块时会关连接，很多教程混着讲

捕获数据库错误 **以后** `sqlite3`

`IntegrityError` 是约束被违反，`OperationalError` 是表/列名写错

```
import sqlite3
conn = sqlite3.connect(":memory:")
conn.execute("CREATE TABLE s(id INTEGER PRIMARY KEY, name TEXT UNIQUE)")
conn.execute("INSERT INTO s VALUES (1, '小明')")
try:
    conn.execute("INSERT INTO s VALUES (2, '小明')")  # name 重复
except sqlite3.IntegrityError as e:
    print("IntegrityError →", e)
try:
    conn.execute("SELECT * FROM 不存在的表")
except sqlite3.OperationalError as e:
    print("OperationalError →", e)
conn.close()
```

实测输出

IntegrityError → UNIQUE constraint failed: s.name
OperationalError → no such table: 不存在的表

📌 爬虫/批量写数据时，重复行不该让整个程序崩

⚠️ 两个异常都继承自 `sqlite3.Error`，图省事可以只 `except sqlite3.Error`；但**不要裸** `except:` —— 会把键盘中断也吞掉

json.dumps 近期 标准库

Python 对象 → JSON 字符串 (接口/存盘都用它)

```
import json
d = {"name": "韩立", "score": 88.5, "tags":
["python", "sql"]}
print(json.dumps(d, ensure_ascii=False))
print(json.dumps(d, ensure_ascii=False,
indent=2))
```

实测输出

```
{"name": "韩立", "score": 88.5, "tags":
["python", "sql"]}
{
  "name": "韩立",
  "score": 88.5,
  "tags": [
    "python",
    "sql"
  ]
}
```

📌 调大模型 API 的请求体; 把结果存成文件

⚠️ `ensure_ascii=False` 不加, 中文会变成 `\u97e9\u7acb`
—— 能用但没看清

json.loads 近期 标准库

JSON 字符串 → Python 对象 (大模型返回的就是个字符串)

```
import json
s = '{"answer": 42, "ok": true, "items": [1, 2]}'
d = json.loads(s)
print(type(d).__name__, d["answer"], d["ok"], d["items"][1])
print("访问不存在的键 →", end=" ")
try:
    d["nope"]
except KeyError as e:
    print("KeyError", e)
```

实测输出

```
dict 42 True 2
访问不存在的键 → KeyError 'nope'
```

📌 解析接口/大模型返回

⚠️ `loads` 吃字符串, `load` 吃文件对象 —— 差一个 s 是两种用法, 报错信息很像 (都叫 `JSONDecodeError`)

json.dump / json.load 直接读写文件 近期

标准库

省掉 open 那一步

```
import json, tempfile, os, shutil
d = tempfile.mkdtemp(); p = os.path.join(d, "out.json")
with open(p, "w", encoding="utf-8") as f:
    json.dump({"n": 3, "who": "韩立"}, f,
ensure_ascii=False)
with open(p, encoding="utf-8") as f:
    print("读回来 →", json.load(f))
shutil.rmtree(d, ignore_errors=True)
```

实测输出

```
读回来 → {'n': 3, 'who': '韩立'}
```

📌 把中间结果存盘, 下次不用重跑

⚠️ 一定要写 `encoding='utf-8'` —— Windows 默认编码不是 utf-8, 中文会乱码或直接 `UnicodeEncodeError`

with open(...) 读文件 近期 标准库

`with` 会自动关文件, 不用手动 close

```
import tempfile, os, shutil
d = tempfile.mkdtemp(); p = os.path.join(d, "a.txt")
open(p, "w", encoding="utf-8").write("第一行\n第二行\n")
with open(p, encoding="utf-8") as f:
    for i, line in enumerate(f, 1):
        print(i, line.rstrip()) #
rstrip 去掉行尾换行
shutil.rmtree(d, ignore_errors=True)
```

实测输出

```
1 第一行
2 第二行
```

📌 读日志/读 txt 语料

⚠️ `line` 自带 `\n`, 直接 print 会多一个空行 —— 要么 `rstrip()`, 要么 print 时加 `end=''`

open(..., 'w') 写文件 近期 标准库

w 覆盖、a 追加，都要显式写编码

```
import tempfile, os, shutil
d = tempfile.mkdtemp(); p = os.path.join(d, "log.txt")
with open(p, "w", encoding="utf-8") as f:
    f.write("第一次\n")
with open(p, "a", encoding="utf-8") as f:
    # a = 追加，不会覆盖
    f.write("第二次\n")
print(open(p, encoding="utf-8").read().strip())
shutil.rmtree(d, ignore_errors=True)
```

实测输出

第一次
第二次

📌 记运行日志 / 导出结果

⚠️ 'w' 会立刻清空原文件 —— 想追加却写了 'w'，上次的结果就没了（且没有提示）

os.path.join / exists 近期 标准库

拼路径永远用 join，别手写 + '/' +

```
import os
p = os.path.join("E:\\项目学习", "交互演示", "函数卡.html")
print(p)
print("存在吗 →", os.path.exists(p))
print("目录名 →", os.path.dirname(p))
print("文件名 →", os.path.basename(p))
print("扩展名 →", os.path.splitext(p)[1])
```

实测输出

E:\项目学习\交互演示\函数卡.html
存在吗 → True
目录名 → E:\项目学习\交互演示
文件名 → 函数卡.html
扩展名 → .html

📌 所有涉及文件路径的代码

⚠️ os.path.join('E:\\a', '/b') 里第二个参数带开头的斜杠时，前面的路径会被整个丢掉 —— 得到 /b

os.makedirs(..., exist_ok=True) 近期 标准库

建多层目录；exist_ok=True 让「已存在」不报错

```
import os, tempfile, shutil
d = tempfile.mkdtemp()
target = os.path.join(d, "导出", "2026-09")
os.makedirs(target, exist_ok=True)
os.makedirs(target, exist_ok=True) # 第二次不报错
print("建好了 →", os.path.isdir(target))
shutil.rmtree(d, ignore_errors=True)
```

实测输出

建好了 → True

📌 写文件前确保目录存在（爬虫存图、导出报告）

⚠️ 不写 exist_ok=True，跑第二遍就 FileExistsError —— 这是「只在第二次运行才炸」的经典 bug

os.listdir / glob 过滤 近期 标准库

列目录，再按后缀筛

```
import os, tempfile, shutil
d = tempfile.mkdtemp()
for n in ["a.txt", "b.txt", "c.csv"]:
    open(os.path.join(d, n), "w").close()
names = os.listdir(d)
print("全部 →", sorted(names))
print("只要 txt →", sorted(n for n in names if n.endswith(".txt")))
shutil.rmtree(d, ignore_errors=True)
```

实测输出

全部 → ['a.txt', 'b.txt', 'c.csv']
只要 txt → ['a.txt', 'b.txt']

📌 批量处理一个文件夹里的文件

⚠️ listdir 给的是文件名不是完整路径 —— 要打开它得再 join 一次（这是最常见的「文件找不到」原因）

`re.findall` 近期 标准库

在一段文本里捞出所有符合模式的片段

```
import re
text = "订单 A1001 金额 88 元; 订单 A1002 金额 192 元"
print("所有订单号 →", re.findall(r"A\d{4}", text))
print("所有数字 →", re.findall(r"\d+", text))
print("成对取 →", re.findall(r"(A\d{4}).*(\d+) 元", text))
```

实测输出

```
所有订单号 → ['A1001', 'A1002']
所有数字 → ['1001', '88', '1002', '192']
成对取 → [('A1001', '88'), ('A1002', '192')]
```

📌 从网页/日志里提取字段

⚠️ 模式字符串前面一定要加 `r` (原始字符串) —— 不加的话 `\d` 会被 Python 先当成转义字符处理

`re.search` + 分组 近期 标准库

只找第一个; 用括号分组把想要的部分单独取出来

```
import re
m = re.search(r"(\d{4})-(\d{2})-(\d{2})", "今天是 2026-09-24 周四")
if m:
    print("整段 →", m.group(0))
    print("年 →", m.group(1), " | 月 →", m.group(2), " | 日 →", m.group(3))
    print("命名分组写法 →", re.search(r"(?P<y>\d{4})", "2026").group("y"))
    print("没找到时 →", re.search(r"zzz", "abc"))
```

实测输出

```
整段 → 2026-09-24
年 → 2026 | 月 → 09 | 日 → 24
命名分组写法 → 2026
没找到时 → None
```

📌 解析日期、版本号、URL 参数

⚠️ `re.search` 找不到时返回 `None`, 直接 `.group()` 会 `AttributeError` —— 先 `if m:`

`re.sub` 近期 标准库

按模式替换 (清洗文本的主力)

```
import re
s = "价格: ¥1,299.00 元 (含税)"
print(re.sub(r"[¥,]", "", s))
print(re.sub(r"\s+", " ", s).strip())
print(re.sub(r"(\d+)\.(\d+)", r"\2.\1", "3.14")) # 分组反向引用
```

实测输出

```
价格: 1299.00 元 (含税)
价格: ¥1,299.00 元 (含税)
14.3
```

📌 清洗爬下来的文本 / 归一化输入

⚠️ `re.sub` 默认替换所有匹配; 只想换第一个要加 `count=1`

`datetime.now()` / `strftime` 近期 标准库

取当前时间, 格式化成字符串

```
from datetime import datetime
now = datetime.now()
print("原样 →", now)
print("常用 →", now.strftime("%Y-%m-%d %H:%M:%S"))
print("文件名用 →", now.strftime("%Y%m%d-%H%M"))
print("星期日历 →", now.strftime("%Y-%m-%d %A"))
```

实测输出

```
原样 → 2026-09-28 14:37:48.303849
常用 → 2026-09-28 14:37:48
文件名用 → 20260928-1437
星期日历 → 2026-09-28 Monday
```

📌 给导出文件起名; 记日志时间戳

⚠️ Windows 上 `strftime('%Y-%m-%d')` 没问题, 但文件名里不能有 `:` —— `%H:%M` 直接拿去当文件名会失败

strptime: 字符串 → 时间 近期 标准库

把文本日期变成能计算的对象（转回来才能减）

```
from datetime import datetime
d = datetime.strptime("2026-09-21", "%Y-%m-%d")
print("解析 →", d, type(d).__name__)
print("换一种格式 →", d.strftime("%Y年%m月%d日"))
try:
    datetime.strptime("2026/09/21", "%Y-%m-%d")
except ValueError as e:
    print("格式对不上 →", e)
```

实测输出

```
解析 → 2026-09-21 00:00:00 datetime
换一种格式 → 2026年09月21日
格式对不上 → time data '2026/09/21' does not
match format '%Y-%m-%d'
```

📌 算复习到期日 / 算实习倒计时

⚠️ 格式串必须和输入**严格一致**（2026/09/21 配 %Y-%m-%d 会报 does not match format）

timedelta: 日期加减 近期 标准库

算差几天、N 天后是哪天

```
from datetime import date, timedelta
anchor = date(2026, 9, 23) # 你的倒推起点
for n in (1, 3, 7, 30):
    print(f"第 {n:>2} 天 →", anchor + timedelta(days=n))
gap = date(2027, 3, 1) - anchor
print("到 2027.03 还有", gap.days, "天 ≈", gap.days // 7, "周")
```

实测输出

```
第 1 天 → 2026-09-24
第 3 天 → 2026-09-26
第 7 天 → 2026-09-30
第 30 天 → 2026-10-23
到 2027.03 还有 159 天 ≈ 22 周
```

📌 1-3-7-30 复习日程；倒推链

⚠️ `date - date` 得到 `timedelta`，取天数用 `.days`；`//7` 是**向下取整**的周数（不是四舍五入）

pathlib.Path 以后 标准库

同一件事的现代写法：`/` 拼路径、`.suffix` 取后缀

```
from pathlib import Path
p = Path("E:/项目学习") / "交互演示" / "函数卡.html"
print(p)
print("后缀 →", p.suffix, " | 名字 →", p.stem, " | 父目录 →", p.parent.name)
print("存在吗 →", p.exists(), " | 是文件吗 →", p.is_file())
```

实测输出

```
E:\项目学习\交互演示\函数卡.html
后缀 → .html | 名字 → 函数卡 | 父目录 → 交互演示
存在吗 → True | 是文件吗 → True
```

📌 新写的脚本（老代码里 `os.path` 也常见，两种都要能读）

⚠️ `Path` 对象不能直接丢给老函数当字符串用——必要时 `str(p)`

collections.Counter 以后 标准库

数词频，一行搞定（比手写字典 +1 干净）

```
from collections import Counter
words = "数据 数据库 数据 模型 数据库 数据".split()
c = Counter(words)
print(c)
print("前三 →", c.most_common(3))
print("'数据' 出现 →", c["数据"], "次")
print("没出现过的词 →", c["不存在"], "次（不报错，返回 0）")
```

实测输出

```
Counter({'数据': 3, '数据库': 2, '模型': 1})
前三 → [('数据', 3), ('数据库', 2), ('模型', 1)]
'数据' 出现 → 3 次
没出现过的词 → 0 次（不报错，返回 0）
```

📌 统计标签分布 / 看数据里哪类最多

⚠️ 取不存在的键返回 **0** 而不是 **KeyError**——好处是省事，坏处是拼错词也照样得 0，不报错

defaultdict(list): 分组 以后 标准库

按 key 把东西塞进不同的桶, 不用先判断 key 在不在

```
from collections import defaultdict
rows = [("讯飞班", "韩立"), ("计科1班", "小红"),
        ("讯飞班", "小明")]
g = defaultdict(list)
for cls, name in rows:
    g[cls].append(name)
print(dict(g))
print("不存在的班 →", g["不存在"], "（自动建了空 list）")
```

实测输出

```
{'讯飞班': ['韩立', '小明'], '计科1班': ['小红']}
不存在的班 → [] （自动建了空 list）
```

🔴 把一堆 (分类 内容) 聚成 (分类: 内容)

⚠️ 读一个不存在的 key 会顺手创建它 —— 遍历时边读边写会报

dictionary changed size during iteration

random.choice / sample / randint 以后 标准库

抽一个、抽几个不重复的、抽一个整数

```
import random
random.seed(42) #
固定种子 → 结果可复现
print("抽一个 →", random.choice(["A", "B", "C"]))
print("抽2个不重复 →", random.sample(range(1, 50), 2))
print("1~6 的整数 →", random.randint(1, 6))
random.shuffle(lst := [1, 2, 3, 4])
print("打乱 →", lst)
```

实测输出

```
抽一个 → C
抽2个不重复 → [8, 2]
1~6 的整数 → 6
打乱 → [2, 4, 1, 3]
```

🔴 造测试数据 / 切分训练集

⚠️ random.seed(42) 是可复现的关键 —— 不设种子, 同样的代码每次都出不同结果, bug 就复现不了

内置 8 张

sorted(key=...) 近期 内置

排序的万能钥匙: key 决定「按什么排」

```
rows = [("小明", 88.5), ("小红", 92.0), ("小刚", 76.5)]
print("按分数 →", sorted(rows, key=lambda r: r[1], reverse=True))
d = {"b": 2, "a": 3, "c": 1}
print("字典按值排 →", sorted(d.items(), key=lambda kv: kv[1]))
print("多级排序 →", sorted(rows, key=lambda r: (-r[1], r[0])))
```

实测输出

```
按分数 → [('小红', 92.0), ('小明', 88.5), ('小刚', 76.5)]
字典按值排 → [('c', 1), ('b', 2), ('a', 3)]
多级排序 → [('小红', 92.0), ('小明', 88.5), ('小刚', 76.5)]
```

🔴 排行榜 / 按时间排日志 / 结果稳定输出

⚠️ key= 收到的是函数, 写 key=r[1] 会直接报错;

list.sort() 是原地改, sorted() 返回新列表

int / float / str 类型转换 近期 内置

输入永远是字符串, 要算就得先转

```
print(int("42") + 1, float("3.5") * 2, str(42) + "!")
print("取整 →", int(3.9), "（截断不是四舍五入） | round →", round(3.9))
print("转不了会炸 →", end=" ")
try:
    int("abc")
except ValueError as e:
    print("ValueError:", e)
```

实测输出

```
43 7.0 42!
取整 → 3 （截断不是四舍五入） | round → 4
转不了会炸 → ValueError: invalid literal for int() with base 10: 'abc'
```

🔴 读用户输入、读文件里的数字

⚠️ int(3.9) 是 3 (截断); 要四舍五入用 round()。

int('3.5') 也会报错, 得先 float()

f-string 格式化 近期 内置

在字符串里直接写变量，还能控制小数位和对齐

```
name, score = "韩立", 88.4567
print(f"{name} 的分数是 {score}")
print(f"保留两位 → {score:.2f}")
print(f"百分比 → {score / 100:.1%}")
print(f"右对齐 → |{name:>8}| | 左对齐 → |{name:<8}| | 补零 → {7:03d}")
```

实测输出

```
韩立 的分数是 88.4567
保留两位 → 88.46
百分比 → 88.5%
右对齐 → |      韩立| | 左对齐 → |韩立      | | 补零 → 007
```

📌 拼日志、拼输出的每一处

⚠️ f-string 只在字符串**前面**有 **f** 时生效；`\n` 在 f-string 里是换行，但**表达式里不能有反斜杠** (3.12 前)

len / range 以后 内置

len 数长度；range 造一串数字（Python 里**从 0 开始、不含结尾**）

```
s = "数据库"
print("len →", len(s), "| range(3) →", list(range(3)), "| range(1,4) →", list(range(1, 4)))
print("倒着数 →", list(range(3, 0, -1)))
for i in range(len(s)):
    print(i, s[i])
```

实测输出

```
len → 3 | range(3) → [0, 1, 2] | range(1,4) → [1, 2, 3]
倒着数 → [3, 2, 1]
0 数
1 据
2 库
```

📌 到处都在用

⚠️ `range(1, 5)` 是 1,2,3,4 (**没有 5**) —— 差一个数的 bug 八成来自这里

enumerate 以后 内置

边遍历边拿到序号，别自己维护计数器

```
for i, name in enumerate(["小明", "小红"], start=1):
    print(i, name)
print("不加 start →", list(enumerate("ab")))
```

实测输出

```
1 小明
2 小红
不加 start → [(0, 'a'), (1, 'b')]
```

📌 要「第几条」的时候

⚠️ `enumerate(x, start=1)` 才是从 1 开始；默认从 0 开始

zip 以后 内置

把两个列表按位置配对，一次遍历两个

```
names = ["小明", "小红", "小刚"]
scores = [88, 92, 76]
for n, s in zip(names, scores):
    print(n, s)
print("配对成字典 →", dict(zip(names, scores)))
print("长度不等时 →", list(zip([1, 2, 3], "ab")))
```

实测输出

```
小明 88
小红 92
小刚 76
配对成字典 → {'小明': 88, '小红': 92, '小刚': 76}
长度不等时 → [(1, 'a'), (2, 'b')]
```

📌 两个来源的数据要一起处理

⚠️ 长度不等时**按短的截断，不报错** —— 数据不齐时会静默丢数据（Python 3.10+ 可用 `strict=True` 让它报错）

sum / min / max 以后 内置

三个聚合内置，都能配 key= 和默认值

```
nums = [3, 1, 4, 1, 5]
print("sum →", sum(nums), " | min →",
      min(nums), " | max →", max(nums))
print("带初值 →", sum(nums, 100))
print("空列表 →", sum([]), end=" / ")
try:
    min([])
except ValueError as e:
    print("min 报错: ", e)
print("按 key 取最大 →", max(["aa", "bbb", "c"],
                              key=len))
```

实测输出

```
sum → 14 | min → 1 | max → 5
带初值 → 114
空列表 → 0 / min 报错: min() iterable argument
is empty
按 key 取最大 → bbb
```

📌 算总分、找最长的那条

⚠️ sum([]) 是 0，但 min([]) / max([]) 会报 ValueError
—— 空列表要先用 default= (3.4+) 或自己判断

any / all 以后 内置

一串条件里「有没有一个真的」 / 「是不是全都真」

```
nums = [2, 4, 6, 8]
print("全是偶数 →", all(n % 2 == 0 for n in
                          nums))
print("有大于 5 的 →", any(n > 5 for n in nums))
print("空列表的默认 →", any([], all([]))
```

实测输出

```
全是偶数 → True
有大于 5 的 → True
空列表的默认 → False True
```

📌 批量校验（检查所有字段都非空）

⚠️ 空列表时 all([]) 是 True（「全都满足」在逻辑上成立）
—— 拿它当「数据没问题」的判断会漏掉空数据的情况

LLM API 15 张

建客户端：两行换一家模型 阶段A LLM API

base_url 指向谁，就是在用谁家的模型。本机 Ollama 也兼容这套接口

```
from openai import OpenAI
client =
OpenAI(base_url="http://127.0.0.1:11434/v1",
      api_key="ollama")
print("base_url =", client.base_url)
```

实测输出

```
base_url = http://127.0.0.1:11434/v1/
```

📌 AI 应用的第一行代码；物联网线的「大模型 API」也是它

⚠️ ⚠️ base_url 里写 127.0.0.1，不要写 localhost ——
Windows 上 localhost 会先解析到 IPv6 ::1，Ollama 的 /v1
在那个地址上连接会被重置（你 09-21 踩过，见 W-28）。
api_key 用本机 Ollama 时随便填，换真厂商时必须换成真的
key

最小一次对话 阶段A LLM API

create 里两个必填：用哪个模型、说什么话

```
r = client.chat.completions.create(
    model=MODEL,
    messages=[{"role": "user", "content": "只回
两个字：你好"}],
    temperature=0)
print(r.choices[0].message.content)
```

实测输出

```
你好
```

📌 所有 AI 功能的原子操作

⚠️ 返回的不是字符串，是对象：正文在

r.choices[0].message.content 里 —— 直接 print(r) 会看到一大坨，这是新手第一个卡点

messages 到底是什么 阶段A LLM API

一个 list, 每个元素是 `{role, content}` 的 dict; role 只有四种

```
msgs = [{"role": "system", "content": "你只回答一个数字, 不要解释"},
        {"role": "user", "content": "1+1 等于几"}]
r =
client.chat.completions.create(model=MODEL,
messages=msgs, temperature=0)
print("类型 =", type(msgs).__name__, "长度 =", len(msgs))
print("角色 =", [m["role"] for m in msgs])
print("回答 =",
r.choices[0].message.content.strip()[:20])
```

实测输出

```
类型 = list 长度 = 2
角色 = ['system', 'user']
回答 = 2
```

🔴 拼上上下文、拼历史、拼检索结果, 全是在往这个 list 里塞东西

⚠️ role 四种: `system` (定规矩) / `user` (人说的) /

`assistant` (模型说的) / `tool` (工具结果)。写错 role 名字不会报错, 模型会当成没看见

system 与 user 的分工 阶段A LLM API

system 定规矩、user 提要求。同一句话放进 system, 行为会变

```
q = "评价一下数据库这门课"
a =
client.chat.completions.create(model=MODEL,
temperature=0, max_tokens=30,
messages=[{"role": "user", "content":
q}]).choices[0].message.content
b =
client.chat.completions.create(model=MODEL,
temperature=0, max_tokens=30,
messages=[{"role": "system",
"content": "只用四个字回答"},
{"role": "user", "content":
q}]).choices[0].message.content
print("无 system: ", a.strip()[:40])
print("有 system: ", b.strip()[:40])
```

实测输出

```
无 system: 数据库是一门非常重要的计算机科学课程, 它主要研究如何有效地存储和管理大量数据的技
有 system: 有趣且重要
```

🔴 控制输出格式/长度/口吻都在这里

⚠️ system 不是必须的, 但它是唯一能稳定约束行为的地方 —— 你 09-21 那个 RAG「装死」问题 (检索对了模型却说没有) 就是提示词过防御写的, 改 system 才好 (W-19)

temperature: 0 和 1 差在哪 阶段A LLM API

同一个问题问两次, 看回答是否稳定

```
q = [{"role": "user", "content": "给这门课起一个名字, 只给名字"}]
for t in (0, 1):
    outs = set()
    for _ in range(2):
        r =
client.chat.completions.create(model=MODEL,
messages=q,

temperature=t, max_tokens=12)

outs.add(r.choices[0].message.content.strip())
print(f"temperature={t} 两次回答不同 =
{len(outs) > 1} {sorted(outs)}")
```

实测输出

```
temperature=0 两次回答不同 = False ['匠心筑梦']
temperature=1 两次回答不同 = True ['智慧探索堂', '智慧探索课']
```

🔴 要可复现的实验 (判分、评测) → 用 0; 要创意/多样 → 调高

⚠️ temperature=0 也不保证逐字相同 —— 只能说「更稳定」。所以任何评测都要跑多次取统计, 不能拿单次结果下结论 (这点在 Agent 消融实验里已经吃过一次)

max_tokens: 把回答截断 阶段A LLM API

限制生成长度; 被截断时 `finish_reason` 会变成 `length`

```
r =
client.chat.completions.create(model=MODEL,
temperature=0, max_tokens=5,
messages=[{"role": "user", "content":
"用三句话介绍杭州"}])
print("内容 =",
repr(r.choices[0].message.content))
print("finish_reason =",
r.choices[0].finish_reason)
```

实测输出

```
内容 = '杭州, 这座美丽的城市'
finish_reason = length
```

🔴 控制成本、控制界面长度

⚠️ `finish_reason='length'` 就是被你自己截断的 (不是模型说完了)。做 JSON 输出时被截断 → `json.loads` 直接炸, 这时要调大 max_tokens 而不是去改解析代码

从响应里该取哪几个字段 阶段A LLM API

正文、结束原因、token 用量 —— 三个最常用的

```
r =
client.chat.completions.create(model=MODEL,
    temperature=0,
    messages=[{"role": "user", "content":
"只回：好"}])
print("正文      =",
r.choices[0].message.content.strip())
print("结束原因  =", r.choices[0].finish_reason)
print("输入 tokens =", r.usage.prompt_tokens, "
输出 tokens =", r.usage.completion_tokens)
```

实测输出

```
正文      = 好
结束原因  = stop
输入 tokens = 33  输出 tokens = 2
```

💡 算成本、排查「为什么断在半句」都靠它

⚠️ `usage` 是按 **token 计费** 的依据。本机 Ollama 免费，但换真厂商后这一行就是钱 —— 养成每次都看一眼的习惯

多轮对话：模型其实没有记忆 阶段A LLM API

它每次都只看到你这次发过去的 messages —— 历史要你自己拼

```
hist = [{"role": "user", "content": "我叫韩立"}]
r1 =
client.chat.completions.create(model=MODEL,
    messages=hist, temperature=0, max_tokens=20)
print("第 1 轮问它我是谁（它不知道）：",

client.chat.completions.create(model=MODEL,
    temperature=0, max_tokens=20,
    messages=hist + [{"role": "user",
"content": "我叫什
么？"}]).choices[0].message.content.strip()
[:24])
hist.append({"role": "assistant", "content":
r1.choices[0].message.content})
hist.append({"role": "user", "content": "我叫什
么？"})
print("第 2 轮先回灌再问：",

client.chat.completions.create(model=MODEL,
    messages=hist, temperature=0,

max_tokens=20).choices[0].message.content.stri
p()[:24])
```

实测输出

```
第 1 轮问它我是谁（它不知道）： 您叫韩立。
第 2 轮先回灌再问： 你叫韩立。
```

💡 做聊天界面的核心机制；也是「上下文怎么变长」的根源

⚠️ 🔑 **模型是无状态的**。忘了回灌 assistant 历史 → 它每一轮都像第一次遇见你。反过来，历史越攒越长 → 迟早爆上下文，所以要裁剪（见 `agent.trim`）

流式输出：一个字一个字吐 阶段A LLM API

`stream=True` 后返回的是一堆 **chunk**，要自己拼起来

```
s =
client.chat.completions.create(model=MODEL,
    temperature=0, stream=True,
    messages=[{"role": "user", "content":
"用三句话介绍杭州，每句以句号结尾。"}])
parts = [ch.choices[0].delta.content for ch in
s if ch.choices[0].delta.content]
print("收到", len(parts), "个 chunk，前 3 个 =",
parts[:3])
print("拼起来 =", "".join(parts)[:40])
```

实测输出

```
收到 20 个 chunk，前 3 个 = ['杭州', '是一座', '历史
悠久']
拼起来 = 杭州是一座历史悠久的城市。西湖以其美丽的风景
而闻名。杭州也是美食和茶文化的中心。
```

💡 做「AI 边想边显示」的界面必须用它（否则用户盯着空白等 10 秒）

⚠️ 流式时**没有** `message.content`，只有 `delta.content`；而且最后一块可能是空串，不加 `if` 判断会拼出一堆 `None`

让模型吐 JSON（然后真解析） 阶段A LLM API

`response_format` 要求 JSON；但模型仍可能加解释文字

```
import json
r =
client.chat.completions.create(model=MODEL,
    temperature=0,
    response_format={"type":
"json_object"},
    messages=[{"role": "user", "content":
"输出 JSON：姓名=韩立，年龄=19"}])
txt = r.choices[0].message.content
print("原始返回 =", txt[:50])
d = json.loads(txt)
print("解析后 =", d, type(d).__name__)
```

实测输出

```
原始返回 = {
  "姓名": "韩立",
  "年龄": 19
}
解析后 = {'姓名': '韩立', '年龄': 19} dict
```

💡 让模型当「结构化数据生成器」—— 你要喂给下一段程序的必经一步

⚠️ 模型经常把 JSON 包在 ``json 里。先 `strip` 掉代码围栏再 `json.loads`，并且永远包一层 `try/except` —— 它会偶尔漏个引号，你不该因此整个程序崩

tools: 让模型「要求调用函数」 阶段A LLM API

你给工具清单，模型决定调哪个、参数是什么——但它不会真的执行

```
tools = [{"type": "function", "function": {
    "name": "get_weather", "description": "查某
    个城市的天气",
    "parameters": {"type": "object",
    "properties": {"city": {"type": "string"}},
    "required": ["city"]}}}]

r =
client.chat.completions.create(model=MODEL,
temperature=0, tools=tools,
    messages=[{"role": "user", "content":
"北京现在几度？用工具查"}])
tc = r.choices[0].message.tool_calls
print("finish_reason =",
r.choices[0].finish_reason)
print("要调的工具 =", None if not tc else
[(x.function.name, x.function.arguments) for x
in tc])
```

实测输出

```
finish_reason = tool_calls
要调的工具 = [('get_weather', '{"city": "北
京"}')]
```

📍 Agent 的地基。物联网线的「工具调用」也是这个

⚠️ 🔑 模型只返回「我想调 get_weather, 参数 {"city": "北京"}」这串东西——真正执行函数的是你的代码。参数是字符串，要 `json.loads` 才是 dict

原始 /api/chat: 为什么还要会这条路 阶段A

LLM API

openai 那条路传不了 `options` (比如 `num_ctx`)，而它决定了你会不会把显存撑爆

```
import requests
r =
requests.post("http://127.0.0.1:11434/api/chat",
    timeout=60, json={
    "model": "qwen2.5:3b", "stream": False,
    "messages": [{"role": "user", "content":
    "只回：收到"}],
    "options": {"temperature": 0, "num_ctx":
    4096}}).json()
print("正文 =", r["message"]["content"])
print("本次输出 token =", r["eval_count"], "| 耗
    时 =", round(r["total_duration"] / 1e9, 2),
    "秒")
```

实测输出

```
正文 = 收到
本次输出 token = 2 | 耗时 = 2.2 秒
```

📍 你的 `agent_上下文消融.py` 走的就是这条；调本机模型参数必须用它

⚠️ ⚠️ `num_ctx` 必须给：Ollama 默认 32768，你 8G 显存撑爆后 runner 会崩，而且崩之后模型照样输出、只是全是垃圾（你实测每道题都答 23——比报错危险得多）

把文本变成向量（bge-m3） 阶段A LLM API

embedding 是语义检索的地基：一段文本 → 1024 个数字

```
e = client.embeddings.create(model="bge-m3",
input="数据库系统概论")
v = e.data[0].embedding
print("维度 =", len(v), "| 前 3 个数 =",
    [round(x, 4) for x in v[:3]])
print("类型 =", type(v).__name__)
```

实测输出

```
维度 = 1024 | 前 3 个数 = [-0.0232, -0.0057,
-0.0558]
类型 = list
```

📍 RAG 的第一步；你 09-21 用 bge-m3 把 Recall@1 做到 87%

⚠️ 向量不是给人看的——单看某个数没有意义，有意义的是两条向量之间的夹角（余弦相似度）。同一段文本每次算出来的向量是一样的（不像生成），所以可以缓存（见 `rag.cache`）

调用会怎么失败：异常类型 + 重试骨架 阶段A

LLM API

连不上、超时、对方返回错误 —— 三种要分开接

```
from openai import APIConnectionError,
APITimeoutError, APIStatusError
def ask(prompt, tries=2):
    for i in range(tries):
        try:
            return
        client.chat.completions.create(model=MODEL,
            temperature=0, timeout=3,
            messages=[{"role": "user",
"content":
prompt}]]).choices[0].message.content
        except (APITimeoutError,
APIConnectionError) as e:
            print(f" 第 {i+1} 次失败:
{type(e).__name__}")
            return None
print("正常调用 =", ask("只回：好")[:10])
bad = OpenAI(base_url="http://127.0.0.1:9/v1",
api_key="x", timeout=2)
try:
    bad.chat.completions.create(model=MODEL,
messages=[{"role": "user", "content": "x"}])
except Exception as e:
    print("打不通的地址会抛 =", type(e).__name__)
```

实测输出

正常调用 = 好
打不通的地址会抛 = APITimeoutError

📌 任何要跑一整晚的脚本（评测、消融）都必须有这层

⚠️ ⚠️ 抄你自己代码里那个坑： `urllib.error.HTTPError` 是 `URLError` 的子类 —— 先接 `URLError` 会把「服务器返回 500」误报成「连不上」，完全误导排查方向

提示词模板：把变量拼进问题里 阶段A LLM API

就是 f-string。但要给模型留出「资料区」，别把指令和资料搅在一起

```
def build(question, docs):
    资料 = "\n".join(f"[{i+1}] {d}" for i, d in
enumerate(docs))
    return (f"只根据下面资料回答，资料里没有就说不知道。
\n\n"
          f"【资料】\n{资料}\n\n【问题】
{question}")
p = build("数据库课讲什么", ["数据库系统概论讲关系模型和
SQL", "高数讲极限"])
print(p)
print("---")
print("模型答：",
client.chat.completions.create(model=MODEL,
temperature=0, max_tokens=30,
messages=[{"role": "user", "content":
p}]]).choices[0].message.content.strip()[:40])
```

实测输出

只根据下面资料回答，资料里没有就说不知道。

【资料】

[1] 数据库系统概论讲关系模型和 SQL
[2] 高数讲极限

【问题】数据库课讲什么

模型答： 数据库课讲关系模型和SQL。根据你提供的资料，数据库系统概论课程内容包括关系模型

📌 RAG 的最后一步；也是「提示词工程」最朴素的形态

⚠️ 给资料编号（[1] [2]）不是装饰 —— 这样你才能要求模型回答时标「依据 [2]」，出问题时能回溯是哪条资料错了

Agent 10 张

工具定义：名字 + 说明 + 参数表 阶段A Agent

description 写得好不好，直接决定模型会不会用对工具

```
import json
def 定义工具(name, desc, props, required):
    return {"type": "function", "function":
        {"name": name, "description": desc,
         "parameters": {"type": "object",
          "properties": props, "required": required}}}
t = 定义工具("search_notes", "在本地学习笔记里搜索
关键词，返回命中的片段",
    {"keyword": {"type": "string",
 "description": "要搜的关键词"}}, ["keyword"])
print(json.dumps(t, ensure_ascii=False,
indent=1))
```

实测输出

```
{
  "type": "function",
  "function": {
    "name": "search_notes",
    "description": "在本地学习笔记里搜索关键词，返回
命中的片段",
    "parameters": {
      "type": "object",
      "properties": {
        "keyword": {
          "type": "string",
          "description": "要搜的关键词"
        }
      },
      "required": [
        "keyword"
      ]
    }
  }
}
```

📌 Agent 的第一块砖；你实验 1-1 里注册的那 4 个工具就是这个结构

⚠️ `description` 是给模型看的（不是注释！）。写「搜索」这种模糊说明，模型就不知道该什么时候用；要写清「什么时候该用、返回什么」

工具分发：名字 → 真函数 阶段A Agent

模型只给名字和参数，中间这层映射得你自己写

```
import json
def 查天气(city):      return f"{city} 晴
26℃"
def 算加法(a, b):      return str(float(a) +
float(b))
TOOLS = {"get_weather": 查天气, "add": 算加法}

def 执行(name, args_json):
    fn = TOOLS.get(name)
    if fn is None:
        return f"没有这个工具: {name}"
    try:
        return fn(**json.loads(args_json))
    except Exception as e:
        return f"工具执行失败:
{type(e).__name__}: {e}"

print(执行("get_weather", '{"city": "合肥"}'))
print(执行("add", '{"a": 1, "b": 41}'))
print(执行("nope", "{}"))
```

实测输出

```
合肥 晴 26℃
42.0
没有这个工具: nope
```

📌 Agent 循环里「模型说要调工具」之后的那一步

⚠️ 🔑 参数是 JSON 字符串，要 `json.loads`；模型还可能给不存在的工具名或缺参数——工具执行失败必须变成一段文字回灌给模型，而不是让程序崩。模型看到「工具执行失败: `KeyError`」会自己换个参数重试

一步工具调用：完整闭环 阶段A Agent

模型要工具 → 你执行 → 把结果回灌 → 模型给最终答案

```
import json
TOOLS = [{"type": "function", "function":
{"name": "get_weather", "description": "查城市天气",
  "parameters": {"type": "object",
"properties": {"city": {"type": "string"}},
"required": ["city"]}}}]
msgs = [{"role": "user", "content": "合肥今天多少度？必须先调工具查，不要猜。"}]
r1 =
client.chat.completions.create(model=MODEL,
messages=msgs, tools=TOOLS, temperature=0)
call = r1.choices[0].message.tool_calls[0]
msgs.append(r1.choices[0].message)
# ★ 必须把 assistant 这条也放回去
msgs.append({"role": "tool", "tool_call_id":
call.id, "content": "合肥 晴 26°C"})
r2 =
client.chat.completions.create(model=MODEL,
messages=msgs, temperature=0, max_tokens=30)
print("工具要的参数 =", call.function.arguments)
print("回灌后最终答 =",
r2.choices[0].message.content.strip()[:30])
```

实测输出

工具要的参数 = {"city": "合肥"}
回灌后最终答 = 今天合肥的温度是26°C。

📌 **Agent 的最小完整闭环**；把这段放进 while 就是 Agent 循环

⚠️ 🔑 **tool** 这条消息必须带 **tool_call_id**，而且前面必须先
把 **assistant**（含 tool_calls）那条塞回历史 —— 少任何一条，
模型都不知道你在回应哪个调用

Agent 循环骨架（带步数上限） 阶段A Agent

while：模型 → 要工具就执行回灌 → 不要工具就结束

```
import json
TOOLS = [{"type": "function", "function":
{"name": "add", "description": "两数相加",
  "parameters": {"type": "object",
"properties": {"a": {"type": "number"}, "b":
{"type": "number"}},
"required": ["a", "b"]}}}]
REAL = {"add": lambda a, b: a + b}
msgs = [{"role": "user", "content": "先算
(17+25)，再把结果加 8。用工具算。"}]
for step in range(1, 5):
  r =
client.chat.completions.create(model=MODEL,
messages=msgs, tools=TOOLS, temperature=0)
  m = r.choices[0].message
  if not m.tool_calls:
    print(f"第 {step} 步：模型给最终答案 =
{m.content.strip()[:40]}")
    break
  msgs.append(m)
  for c in m.tool_calls:
    out = REAL[c.function.name]
    (**json.loads(c.function.arguments))
    print(f"第 {step} 步：调用
{c.function.name}{c.function.arguments} →
{out}")
    msgs.append({"role": "tool",
"tool_call_id": c.id, "content": str(out)})
```

实测输出

第 1 步：调用 add{"a":17,"b":25} → 42
第 1 步：调用 add{"a":42,"b":8} → 50
第 2 步：模型给最终答案 = 首先计算（17+25）得到
42，然后将结果加 8 得到最终结果 50。

📌 **这就是 Agent**：LLM + 循环 + 工具。物联网线的「服务组合」和它同构

⚠️ ⚠️ `for step in range(1, 5)` 那个上限**不是可选项** ——
模型可能反复调同一个工具停不下来，没有上限就是死循环 + 无限调用（真厂商那边就是无限花钱）

消息序列长什么样（画出来）

阶段A

Agent

一次工具调用会让 messages 长出两条：assistant(带 tool_calls) 和 tool

```
import json
TOOLS = [{"type": "function", "function":
{"name": "now", "description": "取当前时间",
"parameters": {"type": "object",
"properties": {}}}}]
msgs = [{"role": "user", "content": "现在几点?
用工具查"}]
r =
client.chat.completions.create(model=MODEL,
messages=msgs, tools=TOOLS, temperature=0)
msgs.append(r.choices[0].message)
msgs.append({"role": "tool", "tool_call_id":
r.choices[0].message.tool_calls[0].id,
"content": "2026-09-28 14:32"})
for i, m in enumerate(msgs, 1):
    role = m["role"] if isinstance(m, dict)
else m.role
    keys = sorted(m.keys()) if isinstance(m,
dict) else "（对象，字段见 assistant 那条）"
    print(f"{i}. role={role} 字段={keys}")
```

实测输出

```
1. role=user 字段=['content', 'role']
2. role=assistant 字段=（对象，字段见 assistant
那条）
3. role=tool 字段=['content', 'role',
'tool_call_id']
```

🔴 调试 Agent 最有效的一招：把 messages 打出来看

⚠️ assistant 那条在代码里是对象、在列表里可以是 dict —— 直接 json.dumps(msgs) 会报「不是 JSON 可序列化」。要排查就 print(m.model_dump())（新版 SDK）

上下文裁剪：聊久了必须扔历史

阶段A

Agent

保 system + 最近 N 条；扔掉的只能扔，不能「压缩」

```
def trim(msgs, keep=3):
    sys = [m for m in msgs if m["role"] ==
"system"]
    rest = [m for m in msgs if m["role"] !=
"system"]
    return sys + rest[-keep:]

hist = [{"role": "system", "content": "你是助
手"}] + [
{"role": "user" if i % 2 else "assistant",
"content": f"第{i}句"} for i in range(1, 11)]
out = trim(hist, keep=3)
print("原长 =", len(hist), "> 裁后 =", len(out))
print("留下 =", [(m["role"], m["content"]) for
m in out])
```

实测输出

```
原长 = 11 -> 裁后 = 4
留下 = [('system', '你是助手'), ('assistant',
'第8句'), ('user', '第9句'), ('assistant', '第10
句')]
```

🔴 长对话、长任务（Agent 跑几十步）迟早撞上

⚠️ ⚠️ 裁剪会切断「工具调用 → 工具结果」的配对（tool 消息的 tool_call_id 找不到对应的 assistant）—— 有些厂商会直接报错。要按「一轮」为单位裁，不要按条数乱切

上下文预算：本机显存是硬约束

阶段A

Agent

同一个问题，给模型多少上下文，直接决定它会不会崩

```
import requests
def ask(num_ctx):
    r =
requests.post("http://127.0.0.1:11434/api/chat",
timeout=120, json={
"model": "qwen2.5:3b", "stream":
False, "options": {"temperature": 0,
"num_ctx": num_ctx},
"messages": [{"role": "user",
"content": "只回：好"}]}.json()
    return r.get("message", {}).get("content",
"（空）")
for n in (512, 4096):
    print(f"num_ctx={n:5d} -> 答 = {ask(n)
[:10]}")
```

实测输出

```
num_ctx= 512 -> 答 = 好
num_ctx= 4096 -> 答 = 好
```

🔴 本机跑模型的必备知识；也是 Agent 消融实验能跑完 100 次调用的原因

⚠️ 🔑 上下文不是越大越好：KV cache 随 num_ctx 线性吃显存。你 8G 显存实测 32768 会崩，4096 之后 30 次突发调用零失败。判断原则：够放「system + 检索资料 + 最近几轮」就行

一次返回多个工具调用 阶段A Agent

模型可以一轮里同时要求调几个工具 —— 你要按顺序都执行完

```
import json
TOOLS = [
    {"type": "function", "function": {"name":
"get_weather", "description": "查城市天气",
    "parameters": {"type": "object",
"properties": {"city": {"type": "string"}},
"required": ["city"]}}},
    {"type": "function", "function": {"name":
"get_time", "description": "查城市当前时间",
    "parameters": {"type": "object",
"properties": {"city": {"type": "string"}},
"required": ["city"]}}}
r =
client.chat.completions.create(model=MODEL,
temperature=0, tools=TOOLS,
    messages=[{"role": "user", "content":
"合肥和杭州的天气、时间都查一下，两个工具都用"}])
tcs = r.choices[0].message.tool_calls or []
print("这一轮要求调", len(tcs), "个工具：",
    [(c.function.name,
json.loads(c.function.arguments)) for c in
tcs])
```

实测输出

```
这一轮要求调 4 个工具： [(('get_weather', {'city':
'合肥'}), ('get_time', {'city': '合肥'}),
('get_weather', {'city': '杭州'}), ('get_time',
{'city': '杭州'})]
```

📌 用户问「A 和 B 都查一下」时就是这个形态

⚠️ 每个 tool_call 都要各自回一条 `role='tool'` 消息（用各自的 id）。只回一条、或漏掉某一个 id → 模型会以为那个工具还没跑完，然后重复要求调用它

ReAct：先写「想法」再决定动作 阶段A Agent

强制模型把推理过程说出来，比直接让它调工具更稳

```
SYS = ("你是一个会用工具的助手。每次回复必须先写一行
「想法：...」（说明你打算怎么做），"
    "再决定是否调用工具。不要跳步。")

r =
client.chat.completions.create(model=MODEL,
temperature=0, max_tokens=60,
    messages=[{"role": "system",
"content": SYS},
    {"role": "user", "content":
"我要把 17 和 25 相加再加 8，你打算分几步做？"}])
print(r.choices[0].message.content.strip()
[:120])
```

实测输出

想法：解决这个问题只需要一步，因为它是一个简单的数学计算问题。我们可以直接计算 $17 + 25 + 8$ 。

想法：调用工具：None，因为这个问题可以通过简单的数学计算来解决。

📌 书上第 1 章的起点思想；调试 Agent 时把想法打出来就知道它哪一步想歪了

⚠️ 「想法」是给人和给下一轮用的中间产物，不是真理 —— 模型写「想法：先查天气」然后调了别的工具，这种不一致很常见，要靠工具结果回灌把它拽回来

把 **Agent** 的状态存下来 阶段A Agent

步数、工具调用次数、trace —— 没有它，出了错你只能瞎猜

```
import json
state = {"steps": 0, "tool_calls": [],
"finished": False}
def step(state, tool_name, result):
    state["steps"] += 1
    state["tool_calls"].append({"step":
state["steps"], "tool": tool_name, "result":
result})
    if state["steps"] >= 3:
        state["finished"] = True
    return state
step(state, "add", 42); step(state, "add", 50)
print(json.dumps(state, ensure_ascii=False,
indent=1))
```

实测输出

```
{
  "steps": 2,
  "tool_calls": [
    {
      "step": 1,
      "tool": "add",
      "result": 42
    },
    {
      "step": 2,
      "tool": "add",
      "result": 50
    }
  ],
  "finished": false
}
```

📌 跑评测（你的消融实验）时靠它记录每一步，而不是只看最后答案

⚠️ 你 09-21 那次「模型崩了却继续输出垃圾、每道题都答 23」能被发现，正是因为留了每次的原始回答。只记总分，会把「蒙对」藏起来（那是 W-26 的教训）

RAG 7 张

切块：长文本要切开才能检索 阶段A RAG

按长度切、留一点重叠 —— 重叠是为了不让一句话被切断在两块里

```
def chunk(text, size=18, overlap=4):
    out, i = [], 0
    while i < len(text):
        out.append(text[i:i + size])
        if i + size >= len(text):
            break
        i += size - overlap
    return out
doc = "数据库系统概论讲关系模型和SQL语言。索引能加快查询速度。事务保证数据一致性。"
for i, c in enumerate(chunk(doc), 1):
    print(f"{i}. {c}")
```

实测输出

1. 数据库系统概论讲关系模型和SQL语言
2. QL语言。索引能加快查询速度。事务保
3. 。事务保证数据一致性。

📌 RAG 第一步。docx/pdf 抽出来的长文本必须切

⚠️ 块切太小 → 检索到的片段缺上下文，模型答不全；切太大 → 一块里混了好几个主题，相似度被稀释。**没有标准答案，要拿你的问题去试**

余弦相似度：两个向量像不像 阶段A RAG

点积 / 模长乘积；结果在 -1~1，越大越像

```
import numpy as np
def cos(a, b):
    a, b = np.asarray(a, dtype=float), np.asarray(b, dtype=float)
    return float(a @ b / (np.linalg.norm(a) * np.linalg.norm(b)))
print("自己 == 自己" ==, round(cos([1, 2, 3], [1, 2, 3]), 4))
print("同方向、长度不同" ==, round(cos([1, 2, 3], [2, 4, 6]), 4))
print("垂直" ==, round(cos([1, 0], [0, 1]), 4))
print("相反" ==, round(cos([1, 0], [-1, 0]), 4))
```

实测输出

自己 == 自己	= 1.0
同方向、长度不同	= 1.0
垂直	= 0.0
相反	= -1.0

📌 所有语义检索的核心运算；你 rag/01 里的排序就是它

⚠️ 余弦只看方向、不看长度 —— 所以文本长短不影响它，这是它比欧氏距离更适合做文本检索的原因。但它对「模型本身好不好」无能为力：embedding 差，余弦再准也搜不到

top-k 检索：排序取前 k 条 阶段A RAG

真实用法：把问题也变成向量，和每条资料算相似度，取最大的 k 个

```
import numpy as np
docs = ["数据库系统概论讲关系模型和SQL", "高数讲极限和导数", "膳食指南建议少油少盐"]
q = "SQL 是什么"
# 这里用字符重合度代替真 embedding，只为演示排序逻辑；真实项目换成 bge-m3
score = [len(set(q) & set(d)) / len(set(q) | set(d)) for d in docs]
order = np.argsort(score)[::-1]
for rank, i in enumerate(order[:2], 1):
    print(f"第{rank}名 相似度={score[i]:.4f} {docs[i]}")
```

实测输出

第1名	相似度=0.1579	数据库系统概论讲关系模型和SQL
第2名	相似度=0.0000	膳食指南建议少油少盐

📌 RAG 的「检索」就是这十行；换掉打分函数就是真语义检索

⚠️ ⚠️ 本卡用字符重合度代替真 embedding，只为你看清排序逻辑 —— 这种「零依赖伪向量」对同义词完全无效（你 09-21 实测：「免疫」「体虚乏力」全部 0 命中，见 W-18）。真项目必须上 bge-m3

TF-IDF：不上模型的检索基线 阶段A RAG

关键词权重检索。做 RAG 时要拿它当对照，才知道语义检索到底赢了多少

```
from sklearn.feature_extraction.text import TfidfVectorizer
docs = ["数据库系统概论讲关系模型和SQL", "高数讲极限和导数", "数据库索引加快查询"]
v = TfidfVectorizer(analyzer="char", ngram_range=(1, 2))
X = v.fit_transform(docs)
print("矩阵形状（文档数 × 特征数）=", X.shape)
q = v.transform(["数据库 索引"])
print("每条得分 =", [round(float(x), 3) for x in (X @ q.T).toarray().ravel()])
```

实测输出

矩阵形状（文档数 × 特征数）=	(3, 53)
每条得分 =	[0.205, 0.08, 0.621]

📌 物联网线的「服务发现」如果不上模型，就是这个；也是论文里常见的 baseline

⚠️ analyzer='char' 是为中文准备的（中文默认按词切会切错）。TF-IDF 完全不理解同义词 —— 搜「索引」不会命中「index」，这是它和语义检索的分水岭

把检索到的资料塞进 **prompt** 阶段A RAG

检索 → 拼提示词 → 生成。这一步错了，前面检索再准也白搭

```
docs = ["数据库系统概论讲关系模型和 SQL 语言", "索引能加快查询速度"]
q = "为什么要建索引"
ctx = "\n".join(f"[{i+1}] {d}" for i, d in enumerate(docs))
prompt = f"只根据下面资料回答：资料里没有就说不知道。
\n{ctx}\n\n问题：{q}"
print("拼好的提示词 ↓")
print(prompt)
print("模型答：",
client.chat.completions.create(model=MODEL,
temperature=0, max_tokens=40,
messages=[{"role": "user", "content":
prompt}]).choices[0].message.content.strip()[:50])
```

实测输出

拼好的提示词 ↓
只根据下面资料回答：资料里没有就说不知道。
[1] 数据库系统概论讲关系模型和 SQL 语言
[2] 索引能加快查询速度

问题：为什么要建索引
模型答：索引能加快查询速度，因为通过在数据库表中创建索引，可以提高数据检索的速度，特别是在需要频繁查询的列上

📌 **RAG 的第三步；也是 rag/02 干的事**

⚠️ ⚠️ **必须写「资料里没有就说不知道」**，否则模型会拿它自己的知识编——那就不叫 RAG 了。但这条又不能写太狠：你 09-21 实测过，过度防御的提示词会让模型「装死」（检索明明命中却说没有，见 W-19）

评测检索质量：**Recall@k** 和 **MRR** 阶段A RAG

没有评测的 RAG 只是「感觉还行」——你那次量化到 Recall@1 87%

```
def recall_at_k(ranks, k):          # ranks: 每条问题的正确资料排在第几名（从 1 开始）
    return sum(1 for r in ranks if r <= k) / len(ranks)
def mrr(ranks):
    return sum(1 / r for r in ranks) / len(ranks)
ranks = [1, 1, 2, 1, 3, 1, 1, 5, 1, 2]      # 10 个问题的正确命中位次
print(f"Recall@1 = {recall_at_k(ranks, 1):.0%}")
Recall@3 = {recall_at_k(ranks, 3):.0%}")
print(f"MRR      = {mrr(ranks):.3f}")
```

实测输出

Recall@1 = 60% Recall@3 = 90%
MRR = 0.753

📌 **要向导师/面试官说「我的检索效果是 X%」时，必须有这个数**

⚠️ **评测集必须是你自己标的**（哪条资料才是正确答案）。用模型自己判断对错 = 自己给自己打分。你的 RAG 那次是手工标的小集，样本量小——报数字时要一起说样本量，别只报百分比

把向量缓存到本地 阶段A RAG

embedding 是确定的，算一次就够了——省时间，也省钱

```
import json, hashlib, tempfile, os, shutil
def key(t):
    return hashlib.md5(t.encode("utf-8")).hexdigest()[:12]
d = tempfile.mkdtemp(); p = os.path.join(d, "cache.json")
docs = ["数据库系统概论", "高数讲义"]
cache = {key(t): [1, 2, 3] for t in docs}
# 假装这里是真的 1024 维向量
with open(p, "w", encoding="utf-8") as f:
    json.dump(cache, f, ensure_ascii=False)
again = json.load(open(p, encoding="utf-8"))
print("缓存键 =", list(again))
print("命中第一条 =", key("数据库系统概论") in again, "| 维度 =", len(again[key("数据库系统概论")]))
shutil.rmtree(d, ignore_errors=True)
```

实测输出

缓存键 = ['a71b1e14fec6', 'c83a669ffc07']
命中第一条 = True | 维度 = 3

📌 **你 rag/_embeddings-cache.json 就是干这个的（170 KB）**

⚠️ 缓存键要用**文本的 hash**，不要用「第几条」——文档一增删，序号全乱，缓存就变成了错答案。换 embedding 模型时**必须清缓存**（不同模型的向量不可比）

pandas 读 CSV 阶段A 数据

两行读进来，形状、列名、前几行立刻能看到

```
import pandas as pd, tempfile, os, shutil
d = tempfile.mkdtemp(); p = os.path.join(d,
"people.csv")
open(p, "w", encoding="utf-8").write("姓名,年龄,
标签\n王奶奶,78,独居\n李爷爷,82,糖尿病\n")
df = pd.read_csv(p, encoding="utf-8")
print("形状（行，列）=", df.shape)
print("列名 =", list(df.columns))
print(df.to_string(index=False))
shutil.rmtree(d, ignore_errors=True)
```

实测输出

```
形状（行，列）= (2, 3)
列名 = ['姓名', '年龄', '标签']
姓名  年龄  标签
王奶奶  78  独居
李爷爷  82  糖尿病
```

📌 物联网线要处理老人数据；任何 Excel/CSV 分析的第一步

⚠️ ⚠️ 中文 CSV 常是 GBK: `encoding='utf-8'` 报 `UnicodeDecodeError` 时换 `'gbk'`。还有 `dtype={'手机号': str}` —— 不然前导 0 和长数字会被 pandas 改掉

选列 / 按条件筛选 阶段A 数据

`df[列名]` 取列、`df[布尔条件]` 取行

```
import pandas as pd
df = pd.DataFrame({"姓名": ["王奶奶", "李爷爷",
"张阿姨"], "年龄": [78, 82, 65],
"标签": ["独居", "糖尿病", "独居"]})
print("只取一列 →", list(df["年龄"]))
print("按条件筛 →")
print(df[df["年龄"] >
70].to_string(index=False))
print("两个条件要用 & 并加括号 →", len(df[(df["年龄"] > 70) & (df["标签"] == "独居")]))
```

实测输出

```
只取一列 → [78, 82, 65]
按条件筛 →
姓名  年龄  标签
王奶奶  78  独居
李爷爷  82  糖尿病
两个条件要用 & 并加括号 → 1
```

📌 清洗数据、挑出要分析的那部分

⚠️ 多条件必须用 `&` `|`，不能用 `and` `or`，而且每个条件都要自己加括号 —— `df[df.a > 1 & df.b < 2]` 不会报错，只会给你错结果（运算符优先级）

groupby: pandas 里的分堆统计 阶段A 数据

和第 7 章 SQL 的 `GROUP BY` 是同一件事，只是换了语言

```
import pandas as pd
df = pd.DataFrame({"标签": ["独居", "糖尿病", "独居", "糖尿病", "独居"],
"年龄": [78, 82, 65, 71, 88]})
g = df.groupby("标签")["年龄"]
print("每组人数 =");
print(g.count().to_string())
print("每组平均 =");
print(g.mean().round(1).to_string())
print("一次算多个 ="); print(g.agg(["count",
"mean", "max"]).to_string())
```

实测输出

```
每组人数 =
标签
独居      3
糖尿病    2
每组平均 =
标签
独居      77.0
糖尿病    76.5
一次算多个 =
      count  mean  max
标签
独居      3   77.0   88
糖尿病    2   76.5   82
```

📌 统计分布；对应你 SQL 里刚学的 `GROUP BY cid, COUNT(*)`

⚠️ `groupby` 默认把分组列变成索引；想让它留成普通列加 `as_index=False` 或 `.reset_index()`。忘了这一步，后面 `merge` 会莫名其妙对不上

merge: pandas 的 JOIN (和第 3 课一模一样)

阶段A 数据

参数就是 left_on / right_on / how —— how 就是 INNER / LEFT

```
import pandas as pd
老人 = pd.DataFrame({"id": [1, 2, 3], "姓名": ["王奶奶", "李爷爷", "张阿姨"]})
需求 = pd.DataFrame({"老人id": [1, 1, 2], "需求": ["助餐", "陪诊", "助浴"]})
inner = 老人.merge(需求, left_on="id", right_on="老人id", how="inner")
left = 老人.merge(需求, left_on="id", right_on="老人id", how="left")
print("inner (两边都配上才留) 行数 =", len(inner))
print(inner[["姓名", "需求"]].to_string(index=False))
print("left (左边一个都不能丢) 行数 =", len(left), "| 没需求的显示 =", left["需求"].isna().sum(), "个 NaN")
```

实测输出

```
inner (两边都配上才留) 行数 = 3
姓名 需求
王奶奶 助餐
王奶奶 陪诊
李爷爷 助浴
left (左边一个都不能丢) 行数 = 4 | 没需求的显示 = 1 个 NaN
```

🔴 🔑 你刚学的 JOIN 直接迁移过来 —— 同一个概念换个语言; 物联网线合并两张表靠它

⚠️ 和 SQL 同一个坑: how='left' 去掉就是 inner, 左边的行会整个消失。另外 pandas 里 NULL 显示成 NaN, 判断要用 isna() 而不是 == None

DataFrame → 喂给模型 / 存盘 阶段A 数据

转成 dict/list 再 json.dumps, 就能塞进 prompt 或发出去

```
import pandas as pd, json
df = pd.DataFrame({"姓名": ["王奶奶", "李爷爷"], "年龄": [78, 82]})
print("转 dict 列表 =", df.to_dict(orient="records"))
print("紧凑 JSON =", df.to_json(orient="records", force_ascii=False))
print("给模型的表格文本 ↓")
print(df.to_csv(index=False, sep="|").strip())
```

实测输出

```
转 dict 列表 = [{"姓名": "王奶奶", "年龄": 78}, {"姓名": "李爷爷", "年龄": 82}]
紧凑 JSON = [{"姓名": "王奶奶", "年龄": 78}, {"姓名": "李爷爷", "年龄": 82}]
给模型的表格文本 ↓
姓名|年龄
王奶奶|78
李爷爷|82
```

🔴 让模型「看」表格数据: 最省 token 的方式是 CSV 文本, 不是 JSON

⚠️ JSON 比 CSV 费 token (每个字段名重复一遍)。行数多的时候优先给 CSV。to_json 默认把中文转成 \uXXXX, 要 force_ascii=False

numpy 向量化: 别写 for 循环算数 阶段A 数据

同样的计算, 向量化版本短得多、快得多

```
import numpy as np, time
a = np.arange(200_000); b = np.arange(200_000)
t = time.perf_counter(); c1 = [x + y for x, y in zip(a.tolist(), b.tolist())]; t1 = time.perf_counter() - t
t = time.perf_counter(); c2 = a + b; t2 = time.perf_counter() - t
print(f"循环 {t1*1000:.1f} ms    向量化 {t2*1000:.1f} ms    快 {t1/t2:.0f} 倍")
print("结果相同 =", c1[:3] == list(c2[:3]))
```

实测输出

```
循环 17.8 ms    向量化 0.7 ms    快 24 倍
结果相同 = True
```

🔴 处理几万条老人数据时, 这是「跑得动」和「跑不动」的区别

⚠️ 向量化不是「更快的循环」, 是「根本不循环」(底层 C 一次算一批)。但遇到「这一步依赖上一步结果」的逻辑, 向量化写不出来 —— 该用循环就用循环

归一化：把向量变成单位长度 阶段A 数据

除以模长。语义检索里提前归一化，后面算相似度就只剩点积

```
import numpy as np
v = np.array([3.0, 4.0])
u = v / np.linalg.norm(v)
print("原向量 =", v, " 模长 =",
      np.linalg.norm(v))
print("单位向量 =", u, " 新模长 =",
      round(float(np.linalg.norm(u)), 6))
print("归一化后 点积 == 余弦 =", round(float(u @
u), 6))
```

实测输出

```
原向量 = [3. 4.]  模长 = 5.0
单位向量 = [0.6 0.8]  新模长 = 1.0
归一化后 点积 == 余弦 = 1.0
```

📌 批量检索时先归一化 → 相似度计算退化成一个矩阵乘法，快很多

⚠️ 零向量不能归一化（除以 0 → nan），检索时会静默排到最后。如果某条资料抽出来是空文本，它的向量就是零向量——要先过滤掉

时间列：字符串 → 真时间 阶段A 数据

`to_datetime` 之后才能排序、算间隔、按月分组

```
import pandas as pd
df = pd.DataFrame({"服务日期": ["2026-03-01",
                                "2026-03-15", "2026-04-02"], "时长": [40, 60,
                                30]})
df["服务日期"] = pd.to_datetime(df["服务日期"])
df["月份"] = df["服务日期"].dt.strftime("%Y-%m")
print(df.to_string(index=False))
print("按月合计时长 ="); print(df.groupby("月份")
["时长"].sum().to_string())
```

实测输出

服务日期	时长	月份
2026-03-01	40	2026-03
2026-03-15	60	2026-03
2026-04-02	30	2026-04
按月合计时长 =		
2026-03	100	
2026-04	30	

📌 时间序列分析；你那 52,482 条真实活动数据必然用得上

⚠️ 不转 `to_datetime` 就 `.dt` 会报错——字符串没有 `.dt`。格式不统一（2026/3/1 混 2026-03-01）时要加 `format='mixed'` 或 `dayfirst=`

FastAPI 7 张

最小应用 + 一个接口 阶段A FastAPI

装饰器写路径，函数返回什么就发什么（dict 自动变 JSON）

```
from fastapi import FastAPI
app = FastAPI()
@app.get("/hello")
def hello():
    return {"msg": "你好"}
print("路径表 =", [r.path for r in app.routes
if r.path.startswith("/")])
print("方法 =", [sorted(r.methods)[0] for r
in app.routes if r.path == "/hello"])
```

实测输出

```
路径表 = ['/openapi.json', '/docs',
'/docs/oauth2-redirect', '/redoc', '/hello']
方法 = ['GET']
```

📌 你 `api.py` 的骨架；服务器 8009 端口上跑的就是这个

⚠️ 返回 dict 就会被自动转成 JSON——你不需要 `json.dumps`。但返回一个不能序列化的对象（比如 datetime 以外的自定义类）会 500

不启服务器也能测接口 阶段A FastAPI

`TestClient` 直接把请求喂给 app——你 27 个测试就是这么跑的

```
from fastapi import FastAPI
from fastapi.testclient import TestClient
app = FastAPI()
@app.get("/add")
def add(a: int, b: int):
    return {"result": a + b}
c = TestClient(app)
r = c.get("/add", params={"a": 17, "b": 25})
print("状态码 =", r.status_code)
print("返回 JSON =", r.json())
bad = c.get("/add", params={"a": "abc", "b":
1})
print("传错类型 → 状态码 =", bad.status_code, "|
详情里有 =", list(bad.json()["detail"]
[0].keys()))
```

实测输出

```
状态码 = 200
返回 JSON = {'result': 42}
传错类型 → 状态码 = 422 | 详情里有 = ['type',
'loc', 'msg', 'input']
```

📌 写测试的正确姿势：不起服务器、不占端口、毫秒级

⚠️ 必填参数没传 → 422 不是 400（FastAPI 的校验错误码是 422）。另外返回的 `r.json()` 才是 dict，`r.text` 是字符串

请求体 + 自动校验 (Pydantic) 阶段A FastAPI

定义模型 → 参数写模型 → 校验、文档、类型全自动

```
from fastapi import FastAPI
from fastapi.testclient import TestClient
from pydantic import BaseModel, Field

class 老人需求(BaseModel):
    老人编号: int = Field(gt=0, description="必须正整数")
    需求: list[str] = Field(min_length=1)
    预算: float | None = None

app = FastAPI()
@app.post("/needs")
def add_need(n: 老人需求):
    return {"收到": n.需求, "条数": len(n.需求)}

c = TestClient(app)
print(c.post("/needs", json={"老人编号": 1, "需求": ["助餐"], "预算": 30.0}).json())
print("需求给空列表 →", c.post("/needs", json={"老人编号": 1, "需求": []}).status_code)
print("编号给 0 →", c.post("/needs", json={"老人编号": 0, "需求": ["助餐"]}).status_code)
```

实测输出

```
{'收到': ['助餐'], '条数': 1}
需求给空列表 → 422
编号给 0 → 422
```

🔴 接口的「守门员」；字段一多，手写校验必然出错

⚠️ 预算: float | None = None 才是可选的；写成 预算: float = None 类型标注就骗人了。另外字段名用中文是可以的 (Python 3 标识符支持)，但发给别人用时会很难受，慎用

主动返回错误: HTTPException 阶段A FastAPI

业务上「不允许」要自己抛，顺便给前端一句人话

```
from fastapi import FastAPI, HTTPException
from fastapi.testclient import TestClient

app = FastAPI()
DB = {1: "王奶奶"}
@app.get("/elder/{eid}")
def get_elder(eid: int):
    if eid not in DB:
        raise HTTPException(status_code=404, detail=f"没有编号为 {eid} 的老人")
    return {"姓名": DB[eid]}

c = TestClient(app)
print("存在 →", c.get("/elder/1").json())
r = c.get("/elder/99")
print("不存在 →", r.status_code, r.json()["detail"])
```

实测输出

```
存在 → {'姓名': '王奶奶'}
不存在 → 404 没有编号为 99 的老人
```

🔴 第 2 课那个 404 排查法讲的就是这个；现在你能造 404 了

⚠️ detail 会被原样发给调用方 —— 不要把堆栈、SQL、路径塞进去 (信息泄露)。想统一改返回格式，用

```
@app.exception_handler
```

路径参数 / 查询参数 / 默认值 阶段A FastAPI

路径里的用 {}, 问号后面的自动变函数参数

```
from fastapi import FastAPI
from fastapi.testclient import TestClient

app = FastAPI()
@app.get("/search")
def search(q: str, limit: int = 5, exact: bool = False):
    return {"查": q, "限制": limit, "精确": exact}

c = TestClient(app)
print("/search?q=助餐 →", c.get("/search", params={"q": "助餐"}).json())
print("/search?q=助餐&limit=2 →", c.get("/search", params={"q": "助餐", "limit": 2}).json())
print("不传 q (必填) → 状态码", c.get("/search").status_code)
```

实测输出

```
/search?q=助餐 → {'查': '助餐', '限制': 5, '精确': False}
/search?q=助餐&limit=2 → {'查': '助餐', '限制': 2, '精确': False}
不传 q (必填) → 状态码 422
```

🔴 所有带筛选/分页的接口

⚠️ limit: int = 5 写在函数签名里就同时完成了三件事：类型转换、校验、默认值 —— 这就是 FastAPI 的价值。不写默认值 = 必填，不传就 422

把它跑起来 (以及线上怎么跑) 阶段A FastAPI

uvicorn 是服务器；本机开发带 reload，线上不带

```
import uvicorn
cfg = uvicorn.Config("main:app", host="127.0.0.1", port=8009, reload=False)
print("host =", cfg.host, "| port =", cfg.port, "| reload =", cfg.reload)
print("app =", cfg.app)
print("（真启动的命令见卡片下方说明；别在脚本里 run —— 会占住终端）")
```

实测输出

```
host = 127.0.0.1 | port = 8009 | reload = False
app = main:app
（真启动的命令见卡片下方说明；别在脚本里 run —— 会占住终端）
```

🔴 你服务器上 8009 端口 + Docker 跑的就是它

⚠️ ⚠️ uvicorn.run(...) 会阻塞住当前进程 (一直跑服务器)，所以它只该出现在 if __name__ == "__main__": 里 —— 否则你 import 这个模块时测试就卡死了。线上要关掉 reload (它靠监控文件变化工作，占资源且不安全)

开发时: uvicorn main:app --reload --port 8009 | 线上: uvicorn main:app --host 0.0.0.0 --port 8009 (去掉 reload)

流式返回：把模型的字一个个传给前端 阶段A

FastAPI

`StreamingResponse` + 生成器；配 `llm.stream` 就是「AI 边写边显示」

```
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
from fastapi.testclient import TestClient
app = FastAPI()
def gen():
    for piece in ["数据", "库", "很", "有用"]:
        yield piece
@app.get("/chat")
def chat():
    return StreamingResponse(gen(),
media_type="text/plain; charset=utf-8")
r = TestClient(app).get("/chat")
print("状态码 =", r.status_code, "| Content-Type =", r.headers["content-type"])
print("拼起来 =", r.text)
```

实测输出

```
状态码 = 200 | Content-Type = text/plain; charset=utf-8
拼起来 = 数据库很有用
```

📌 做 AI 聊天界面的必备件（否则用户要等整段生成完才看到字）

⚠️ 生成器里 `yield` 的是字符串/字节，不能 `yield dict`。而且流式一开，就没法再改状态码了——出错只能在流里发一个约定的错误标记

第三方 2 张

`requests.get`（只看签名） 以后 第三方

HTTP 请求最常用的库。这条**只实测了签名**，没联网

```
import inspect, requests
print("requests", requests.__version__)
print("get", inspect.signature(requests.get))
print("post 有 params/data/json →",
      all(k in
inspect.signature(requests.post).parameters
for k in ("params", "data", "json")))
```

实测输出

```
requests 2.32.5
get (url, params=None, **kwargs)
post 有 params/data/json → False
```

📌 之后调接口 / 爬虫 / 调大模型 API

⚠️ ⚠️ 这段**没有真的发请求**（不允许联网实测）。用法要点：

`resp.status_code` 判断成败、`resp.json()` 取 JSON、`resp.text` 取原文、`timeout=` 一定要给（不给会永久挂住）
[外源] 未联网验证

`pandas.read_csv` (只看签名) 以后 第三方

读表格数据的事实标准。同样**只实测了签名**

```
import inspect, pandas as pd
print("pandas", pd.__version__)
print("read_csv",
inspect.signature(pd.read_csv))
df = pd.DataFrame({"a": [1, 2], "b": ["x",
"y"]})
print(df.to_string(index=False))
```

实测输出

```
pandas 2.3.3
read_csv (filepath_or_buffer: 'FilePath |
ReadCsvBuffer[bytes] | ReadCsvBuffer[str]', *,
sep: 'str | None | lib.NoDefault' =
<no_default>, delimiter: 'str | None |
lib.NoDefault' = None, header: "int |
Sequence[int] | None | Literal['infer']" =
'infer', names: 'Sequence[Hashable] | None |
lib.NoDefault' = <no_default>, index_col:
'IndexLabel | Literal[False] | None' = None,
usecols: 'UsecolsArgType' = None, dtype:
'DtypeArg | None' = None, engine: 'CSVEngine |
None' = None, converters: 'Mapping[Hashable,
Callable] | None' = None, true_values: 'list |
None' = None, false_values: 'list | None' =
None, skipinitialspace: 'bool' = False,
skiprows: 'list[int] | int |
Callable[[Hashable], bool] | None' = None,
skipfooter: 'int' = 0, nrows: 'int | None' =
None, na_values: 'Hashable |
Iterable[Hashable] | Mapping[Hashable,
Iterable[Hashable]] | None' = None,
keep_default_na: 'bool' = True, na_filter:
'bool' = True, verbose: 'bool | lib.NoDefault'
= <no_default>, skip_blank_lines: 'bool' =
True, parse_dates: 'bool | Sequence[Hashable]
| None' = None, infer_datetime_format: 'bool |
lib.NoDefault' = <no_default>, keep_date_col:
'bool | lib.NoDefault' = <no_default>,
date_parser: 'Callable | lib.NoDefault' =
<no_default>, date_format: 'str |
dict[Hashable, str] | None' = None, dayfirst:
'bool' = False, cache_dates: 'bool' = True,
iterator: 'bool' = False, chunksize: 'int |
None' = None, compression:
'CompressionOptions' = 'infer', thousands:
'str | None' = None, decimal: 'str' = '.',
lineterminator: 'str | None' = None,
quotechar: 'str' = '"', quoting: 'int' = 0,
doublequote: 'bool' = True, escapechar: 'str |
None' = None, comment: 'str | None' = None,
encoding: 'str | None' = None,
encoding_errors: 'str | None' = 'strict',
dialect: 'str | csv.Dialect | None' = None,
on_bad_lines: 'str' = 'error',
delim_whitespace: 'bool | lib.NoDefault' =
<no_default>, low_memory: 'bool' = True,
memory_map: 'bool' = False, float_precision:
"Literal['high', 'legacy'] | None" = None,
storage_options: 'StorageOptions | None' =
```

```
None, dtype_backend: 'DtypeBackend |
lib.NoDefault' = <no_default>) -> 'DataFrame |
TextFileReader'
a b
1 x
2 y
```

📌 之后做数据分析 / 处理表格数据

⚠️ ⚠️ 只验证了 DataFrame 构造, **没有真读 csv 文件**。要点:
`encoding='utf-8'` (中文 csv 常是 `gbk`)、`dtype=` 防手机
号被当数字

[外源] 未读实际文件

数据 `_函数卡数据.js` (由 `tools/build_fn_cards.py` 实测生成, 别手改) | 主题 `_主题.css` | 重跑: `python tools/build_fn_cards.py`